

Code-flowchart drill-downs

Open a scenario box in the execution handbook to reach these deeper graphical diagrams. Each code card contains numbered literal source lines; bold main paths and lighter supporting paths distinguish the selected behavior from prerequisites. Called helpers can link to another diagram.

These are selected captured statements, not full function bodies or recorded traces. Complete implementations remain in the source explorer and implementation PDF. Long diagrams continue through explicit linked arrows across parts; no source line is shortened or replaced with an ellipsis.

Source font: DocmapCodeMono, 9.2 pt. Tabs expand to four spaces; unsupported source glyphs use explicit Unicode escapes. Snapshot 2026-10-04T23:16:35-05:00; generated on request and may be stale.

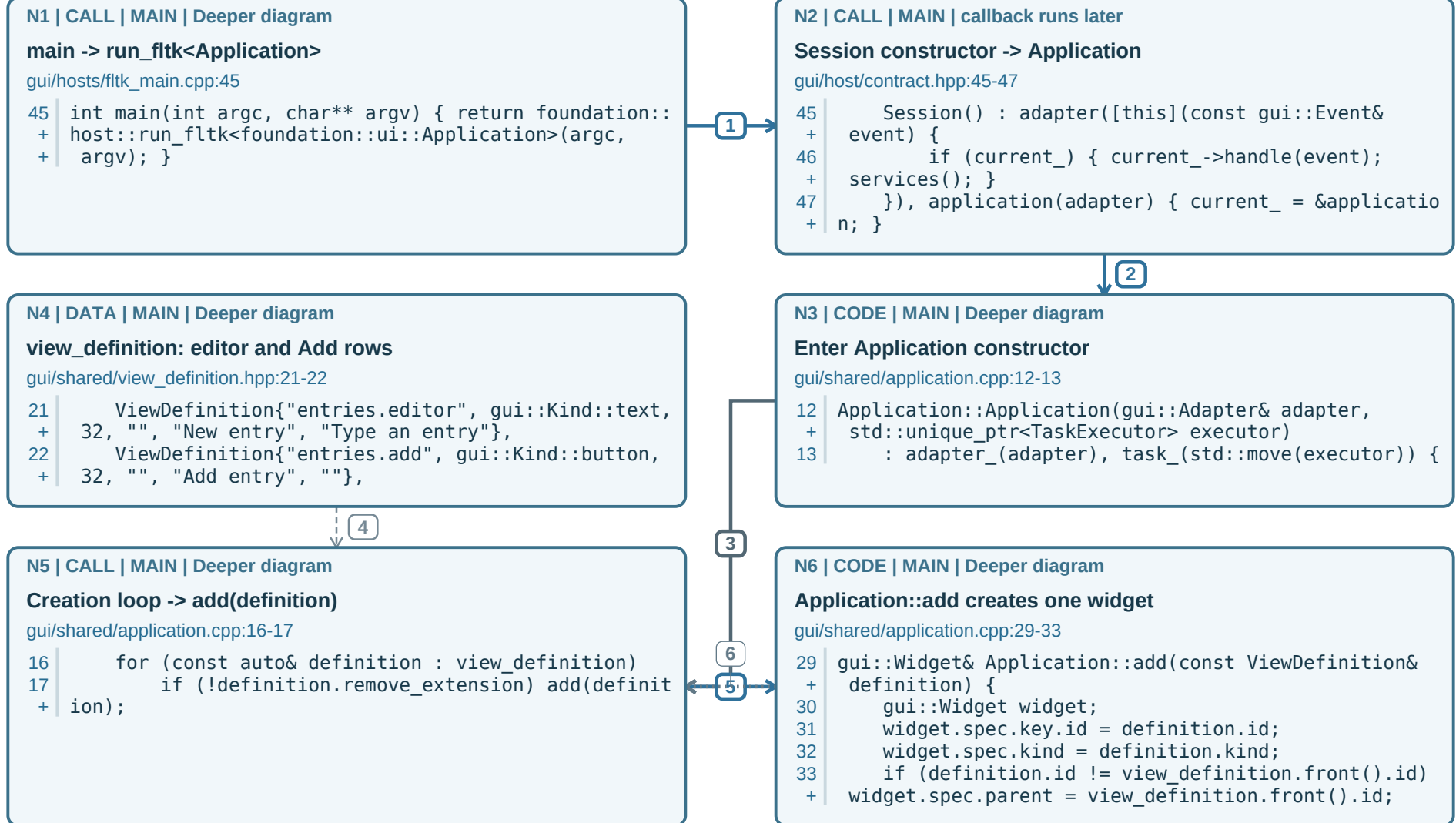
Start here: [main\(\)](#) to widget definitions. Follow the startup calls into the application constructor, widget declarations, and widget-creation loop.

Detailed code diagram	Scope	Parts
main to the declared widgets: calls and the data they consume	Primary native startup to shared Snapshot widget creation	1
Widget declaration table: fields, rows and their consumer	Declaration data and constructor consumer	2
Shell entry, Python dispatch and process plumbing	Literal wrapper entry and representative non-Windows subprocess boundary.	2
execute: configure, select, compile and dispatch	Wrapper statement order; not a configured target execution trace.	2
The wrapper rechecks inputs before its success/error exit	Real terminal source checks and representative input/failure boundaries; omitted guards remain in source.	2
CMake declarations: provider, core sources and CLI link	Configuration declarations and target relationships; no compile/link chronological ordering.	2
CMake selects and declares the Rust component	Configure-time commands versus deferred build-target commands.	2
Rust custom target enters the frozen Cargo builder	Actual builder code and the separate generated verification command.	2
Configured test commands and their compile prerequisites	Configured declarations/prerequisites; actual child entry code is one level below.	2
core.store enters the C++ contract test	Representative existing C++ test statements, with omitted assertions named explicitly.	2
tools.* enters the recorded unittest suite runner	Existing Python suite entry, dynamic module selection and actual unittest call.	2
Manual and hosted release entry boundaries	Manual alternative plus hosted scheduling declarations; local work still runs in preparation-only mode.	2
Prepared producer: full test, package, archive verification	Actual prepared producer and its package subprocess; not candidate publication.	2
Assemble complete local releases from existing archives	Actual local helper calls/statements with staging and loop steps explicitly omitted.	2
Candidate publication has a plan branch and an execute branch	Actual remote mutation branch; prerelease publication does not promote Latest.	2

Detailed code diagram	Scope	Parts
Qualify exact bytes, calculate eligibility and attach an attempt	Grouped physical execution, exact certificate calculation and separate immutable attachment boundary.	2
Exact certificate verification precedes the Latest mutation	Actual promotion branch plus separately scheduled final verification.	2
ViewDefinition: literal types and declared defaults	Application-owned declaration types and defaults	1
Native GUI startup: real entry and host calls	Native composition; supporting host infrastructure	2
Object relationships: Session, Application and Store	Object ownership and declaration relationships	2
Application constructor: declarations to first publish	Shared application behavior	2
Application::add: create one declared widget	Shared application behavior	1
Add-button dispatch: callback to helper and publication	Shared application behavior	1
append_entry: epoch, core call, success or caught error	Shared application behavior	2
Store::add: validate, capacity, identity, insertion	Reusable domain behavior	1
Store::validate: ABI status to C++ contract	Reusable domain validation	1
Private ABI bridge and the selected C++ provider	Private provider boundary	2
Selected Rust provider: preflight, borrow, pure checks	Private Rust provider alternative	1
Publication: authoritative records to adapter, then commit	Shared application behavior	1
Shared layout: rows, composition callback, bounds and clip	Shared application behavior	2
Menu service request: fields, export bound, enqueue	Shared application behavior	2
Native service pump: poll, begin, dispatch, later reply	Supporting native service infrastructure	2
complete_service: correlation, outcome, parser, publication	Shared application behavior	2
Import parser: staged Store loop and atomic commit	Shared application behavior	2
Native content service: selector, worker, transfer and UI poll	Supporting native file-service infrastructure	2

[Up: execution scenario chooser](#) | [Implementation listings](#) | [Compiler declarations](#)

Up: startup | Evidence and other parents | Interactive diagram



1. N1 -> N2 [call] run_fltk runner creates NativeSession; Session constructor

3. N3 -> N5 [step] constructor body; executor check and title set

5. N5 -> N6 [call] eligible row: add(definition)

2. N2 -> N3 [call] application(adapter)

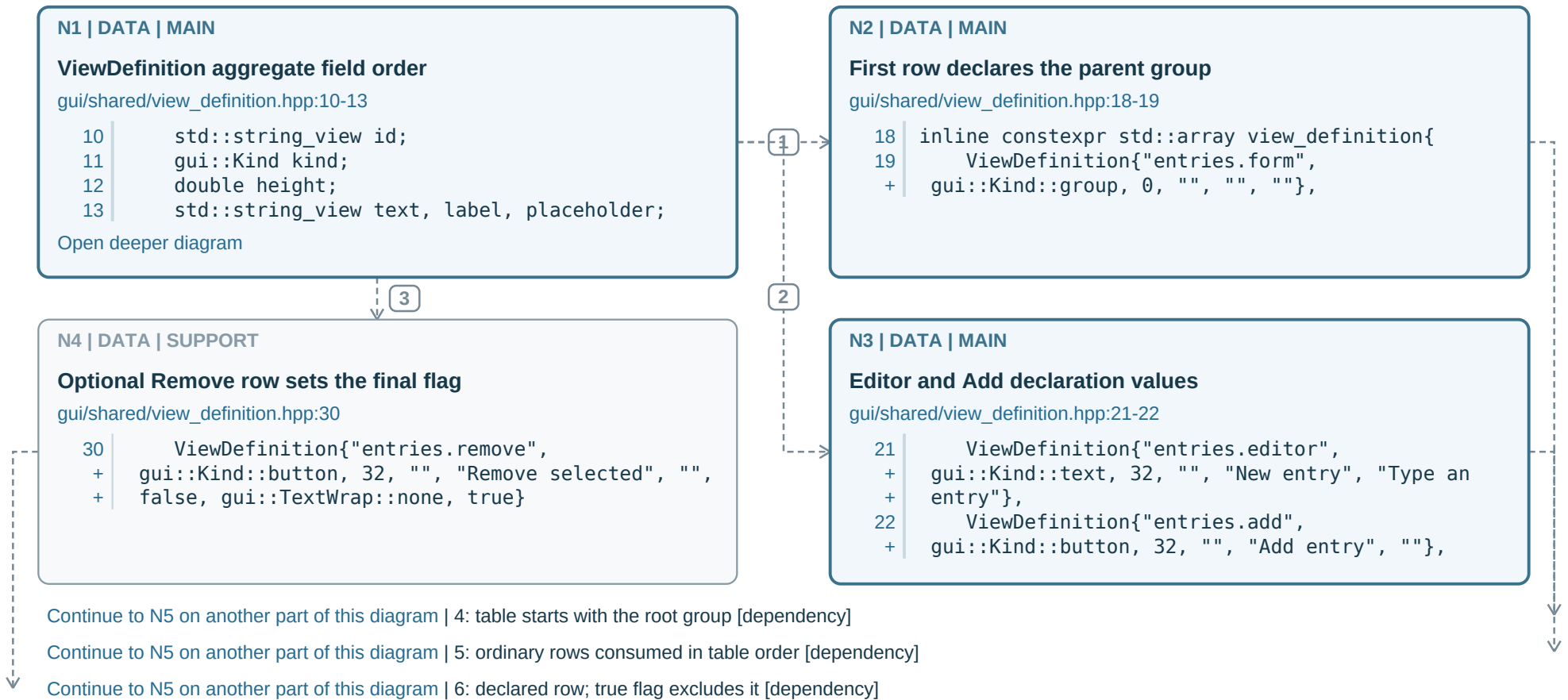
4. N4 -> N5 [dependency] view_definition supplies declaration rows

6. N6 -> N5 [return] add returns; next declaration row

Calls follow startup; DATA is a declaration dependency. Session callback runs later. '+' wraps source; click cards for deeper diagrams.

Up: [startup](#) | [Evidence and other parents](#) | [Interactive diagram](#)

Diagram part 1 of 2 | [Next part](#)



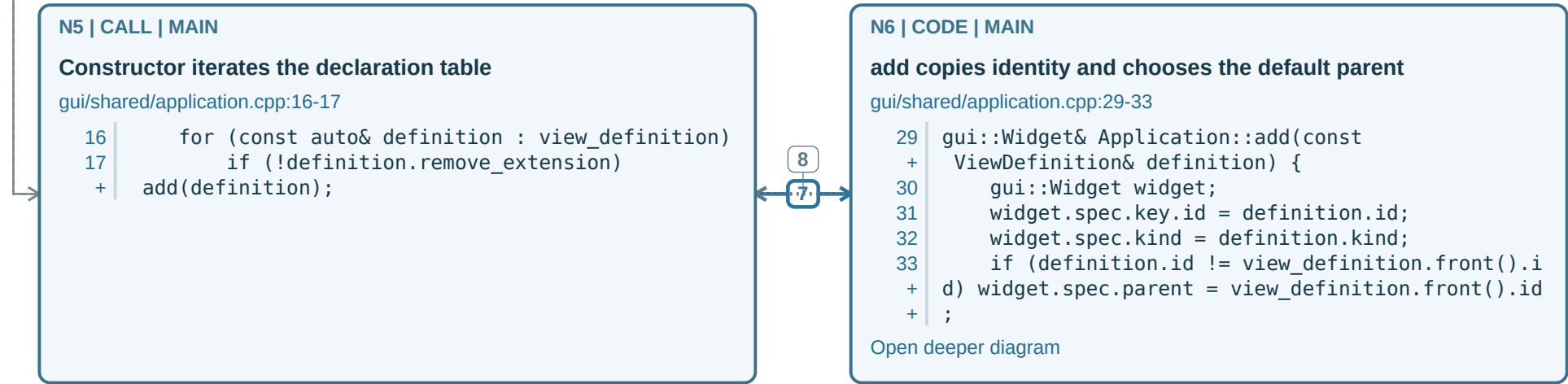
- 1. N1 -> N2 [dependency] aggregate field order determines row meanings
- 2. N1 -> N3 [dependency] same fields and declared defaults
- 3. N1 -> N4 [dependency] final field is remove_extension

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: startup | Evidence and other parents | Interactive diagram

Diagram part 2 of 2 | Previous part

- From N2 on another part of this diagram | 4: table starts with the root group [dependency]
- From N3 on another part of this diagram | 5: ordinary rows consumed in table order [dependency]
- From N4 on another part of this diagram | 6: declared row; true flag excludes it [dependency]

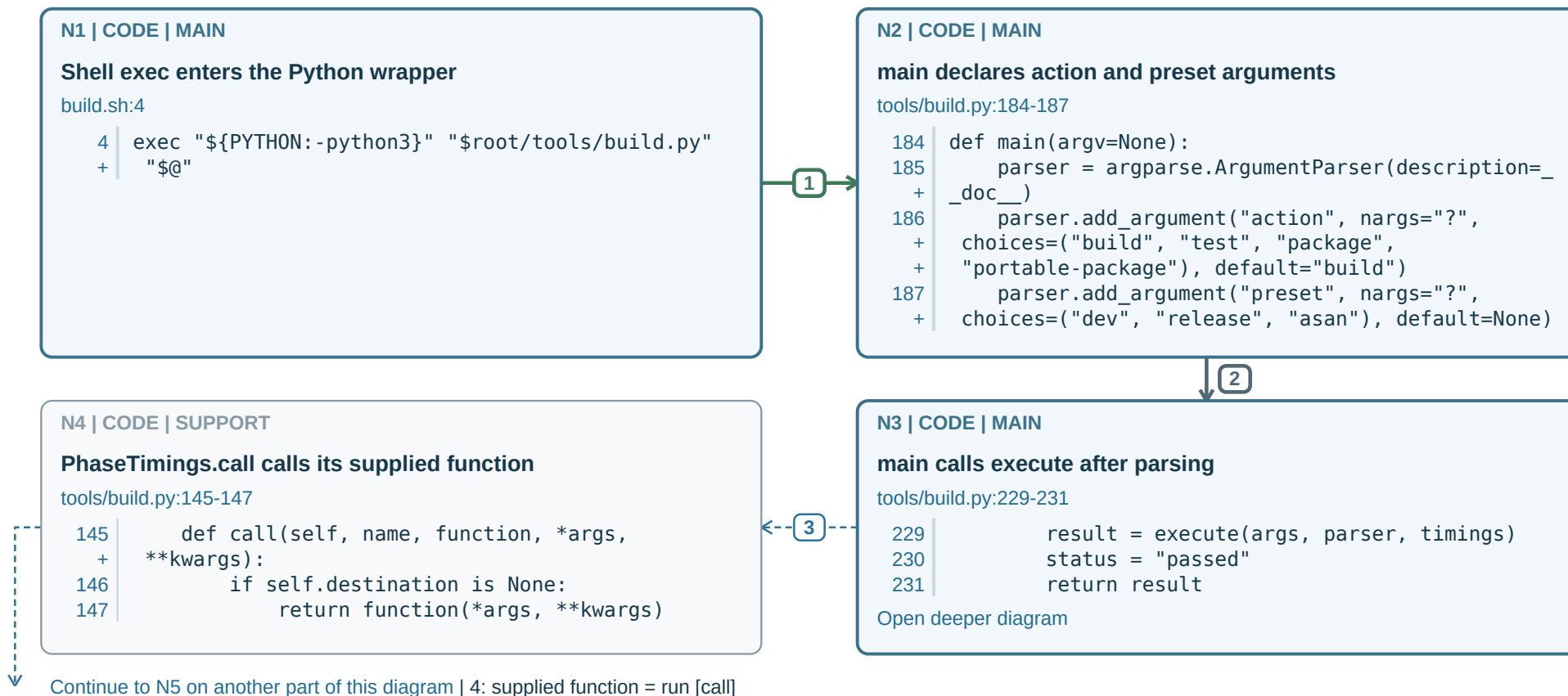


- 7. N5 -> N6 [call] remove_extension is false: add(definition)
- 8. N6 -> N5 [return] return; continue iteration

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [compile](#) | [Evidence and other parents](#) | [Interactive diagram](#)

Diagram part 1 of 2 | [Next part](#)



1. N1 -> N2 [process] exec Python
2. N2 -> N3 [step] parse options; then invoke
3. N3 -> N4 [call] via execute command calls

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

[Up: compile | Evidence and other parents | Interactive diagram](#)

[Diagram part 2 of 2 | Previous part](#)

[From N4 on another part of this diagram | 4: supplied function = run \[call\]](#)

N5 | [CODE](#) | [SUPPORT](#)

run delegates to the platform wrapper

tools/build.py:132-134

→ 132 def run(command, **kwargs):
133 print("+ " + subprocess.list2cmdline([str(x)
+ for x in command]), flush=True)
134 windows_compiler.run(command, cwd=R00T,
+ **kwargs)

--- 5 ---→

N6 | [CODE](#) | [SUPPORT](#)

Non-Windows branch launches the child process

tools/windows_compiler.py:155-156

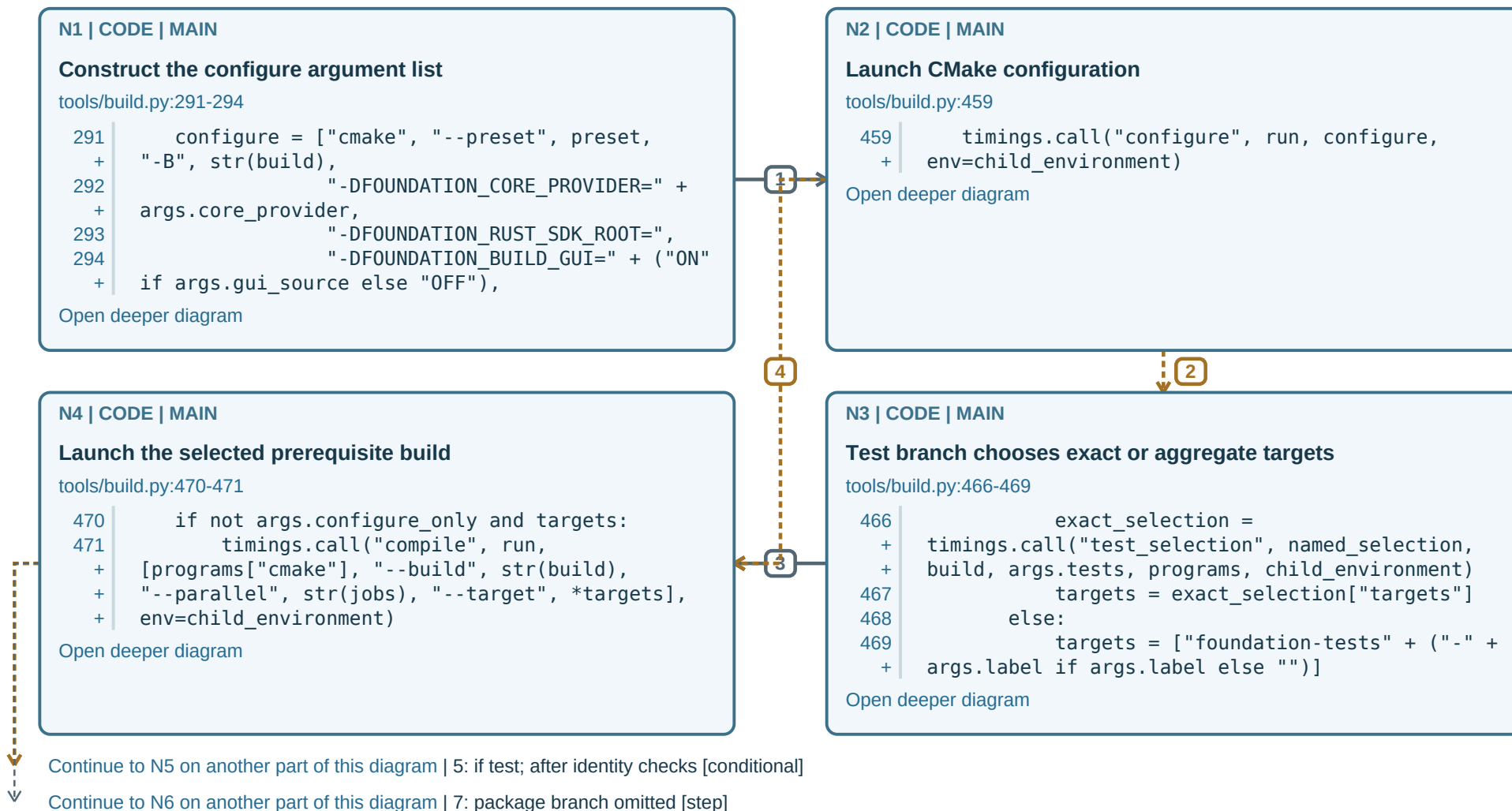
155 if os.name != 'nt':
156 return subprocess.run(command, cwd=cwd,
+ env=env, check=True)

5. N5 -> N6 [conditional] platform wrapper; if not Windows

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: compile | Evidence and other parents | Interactive diagram

Diagram part 1 of 2 | Next part

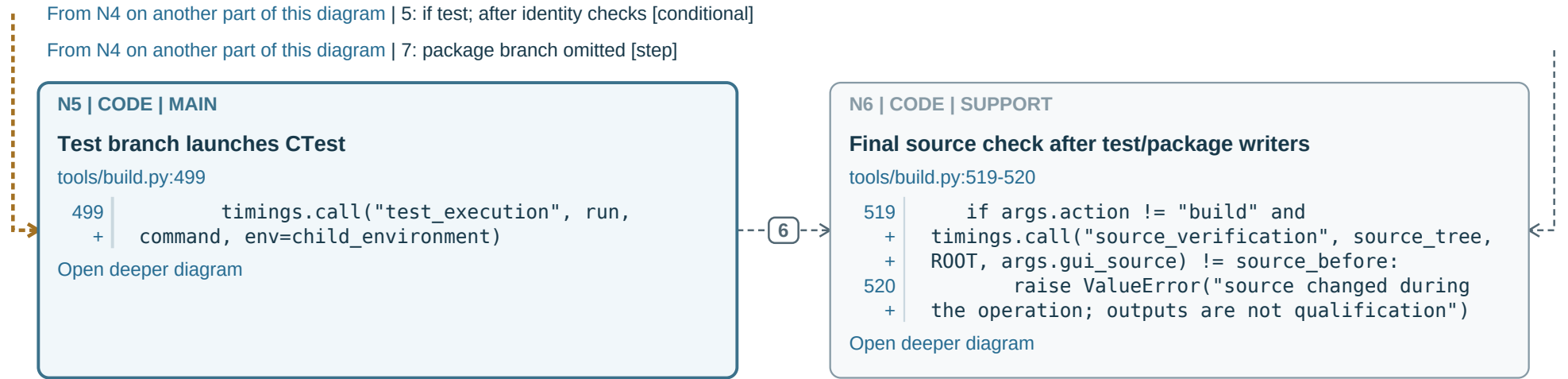


1. N1 -> N2 [step] after input checks and stamps
2. N2 -> N3 [conditional] back in execute: if test
3. N3 -> N4 [step] selected prerequisites
4. N2 -> N4 [conditional] plain build/package: all

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: compile | Evidence and other parents | Interactive diagram

Diagram part 2 of 2 | Previous part

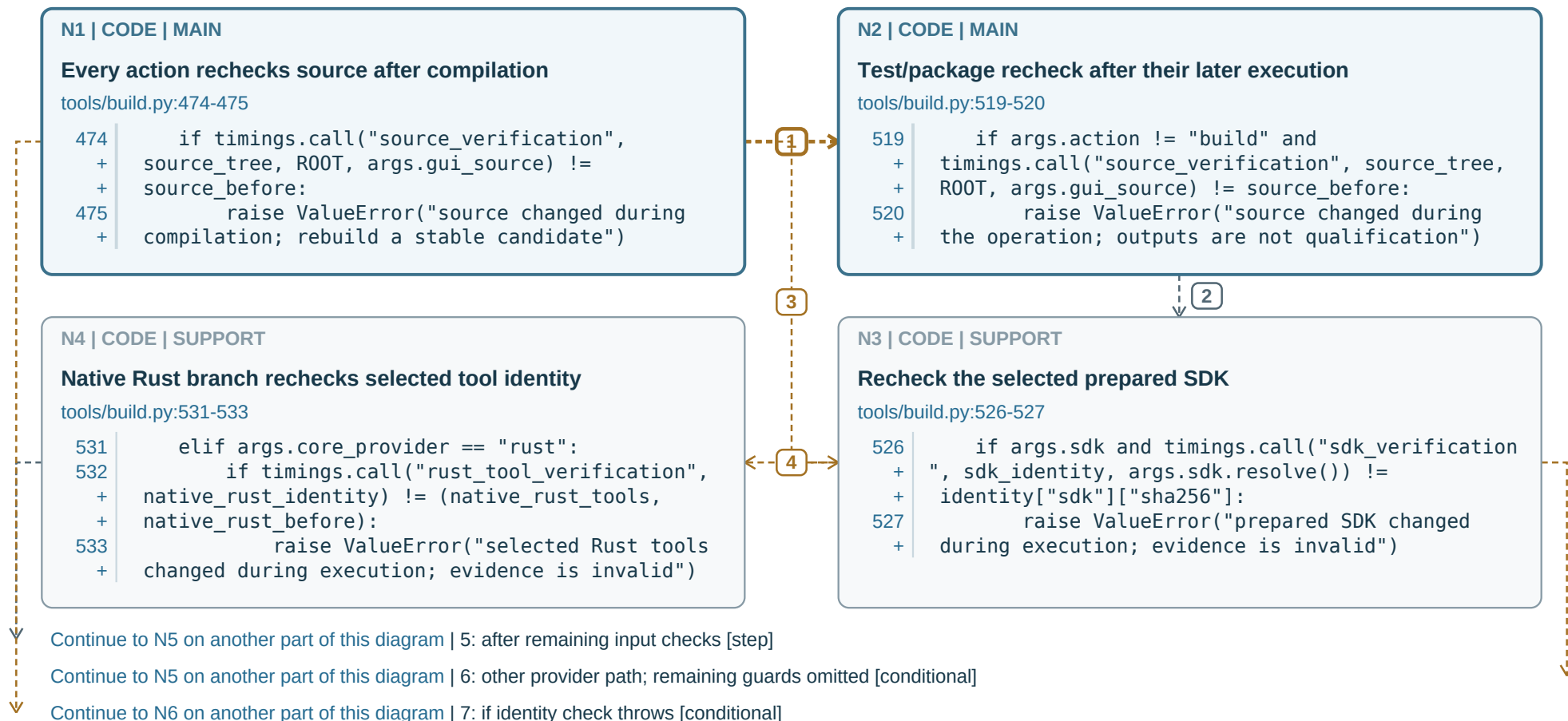


6. N5 -> N6 [step] after test return

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: compile | Evidence and other parents | Interactive diagram

Diagram part 1 of 2 | Next part



1. N1 -> N2 [conditional] after test/package execution
2. N2 -> N3 [step] source unchanged; other selected checks
3. N1 -> N3 [conditional] plain build skips second source check
4. N3 -> N4 [conditional] if native Rust tools selected

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

[Up: compile](#) | [Evidence and other parents](#) | [Interactive diagram](#)

[Diagram part 2 of 2](#) | [Previous part](#)

[From N4 on another part of this diagram](#) | 5: after remaining input checks [step]

[From N3 on another part of this diagram](#) | 6: other provider path; remaining guards omitted [conditional]

[From N1 on another part of this diagram](#) | 7: if identity check throws [conditional]

N5 | CODE | MAIN

execute returns zero after all selected checks

tools/build.py:542-545

```
542     return 0
543
544
545 if __name__ == "__main__":
```

N6 | CODE | SUPPORT

CLI boundary reports failures and returns exit 1

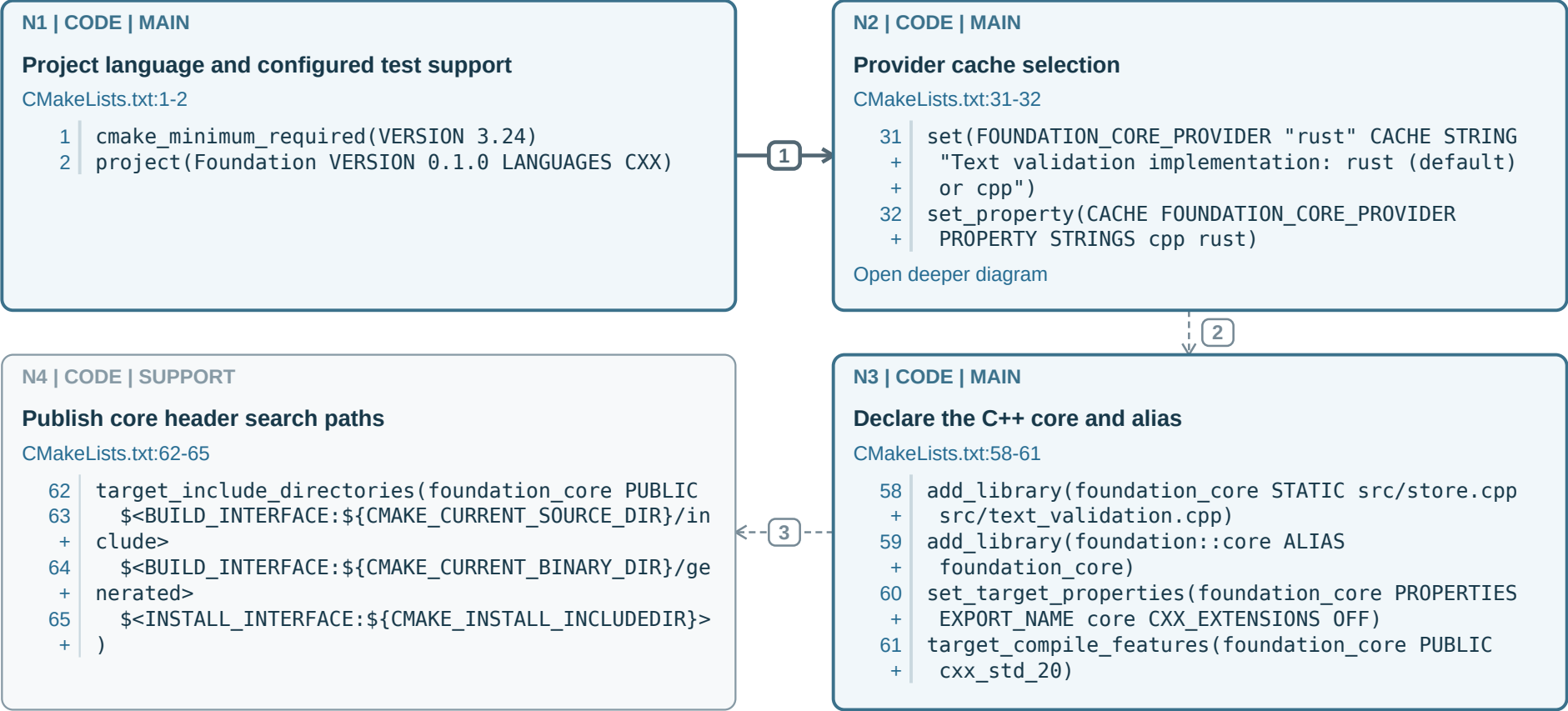
tools/build.py:548-550

```
548     except (OSError, ValueError, KeyError,
+ subprocess.CalledProcessError) as error:
549         print("build: " + str(error),
+ file=sys.stderr)
550         sys.exit(1)
```

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: compile | Evidence and other parents | Interactive diagram

Diagram part 1 of 2 | Next part



Continue to N5 on another part of this diagram | 4: CLI links foundation::core [dependency]

Continue to N6 on another part of this diagram | 5: selected value enters helper [dependency]

Continue to N6 on another part of this diagram | 6: core target argument [dependency]

- 1. N1 -> N2 [step] configuration continues; intervening options omitted
- 2. N2 -> N3 [dependency] provider does not replace the C++ Store
- 3. N3 -> N4 [dependency] public include properties

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

[Up: compile](#) | [Evidence and other parents](#) | [Interactive diagram](#)

[Diagram part 2 of 2](#) | [Previous part](#)

[From N3 on another part of this diagram](#) | 4: CLI links foundation::core [dependency]

[From N2 on another part of this diagram](#) | 5: selected value enters helper [dependency]

[From N3 on another part of this diagram](#) | 6: core target argument [dependency]

N5 | CODE | MAIN

Declare CLI sources and its core link

CMakeLists.txt:105-108

```
105 add_executable(foundation-cli src/main.cpp)
106 target_link_libraries(foundation-cli PRIVATE
+   foundation::core)
107 target_include_directories(foundation-cli PRIVATE
+   "${CMAKE_CURRENT_BINARY_DIR}/generated")
108 foundation_options(foundation-cli)
```

N6 | CODE | MAIN

Call the provider-specific configuration helper

CMakeLists.txt:117-119

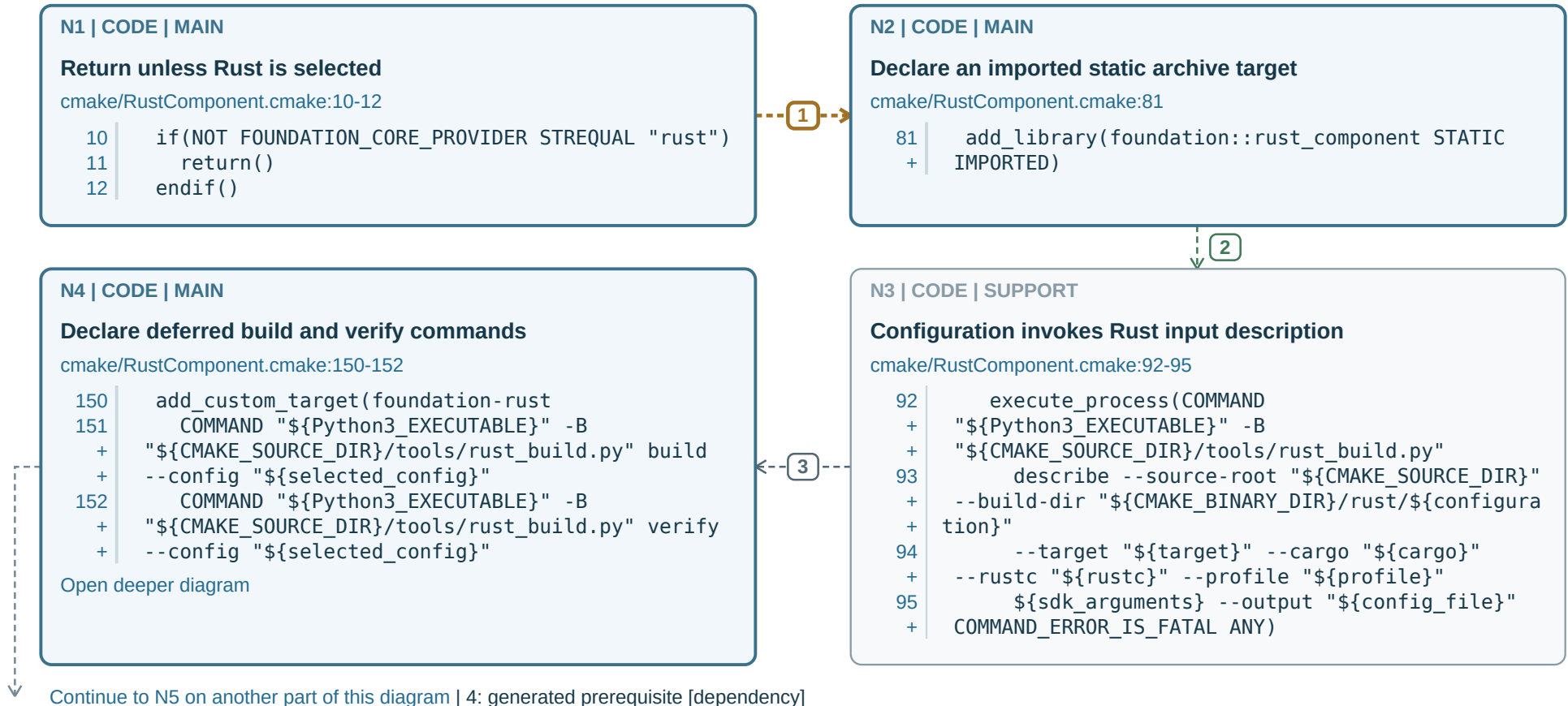
```
117 find_package(Python3 3.9 REQUIRED COMPONENTS
+   Interpreter)
118 include(cmake/RustComponent.cmake)
119 foundation_rust_component(foundation_core)
```

[Open deeper diagram](#)

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: compile | Evidence and other parents | Interactive diagram

Diagram part 1 of 2 | Next part

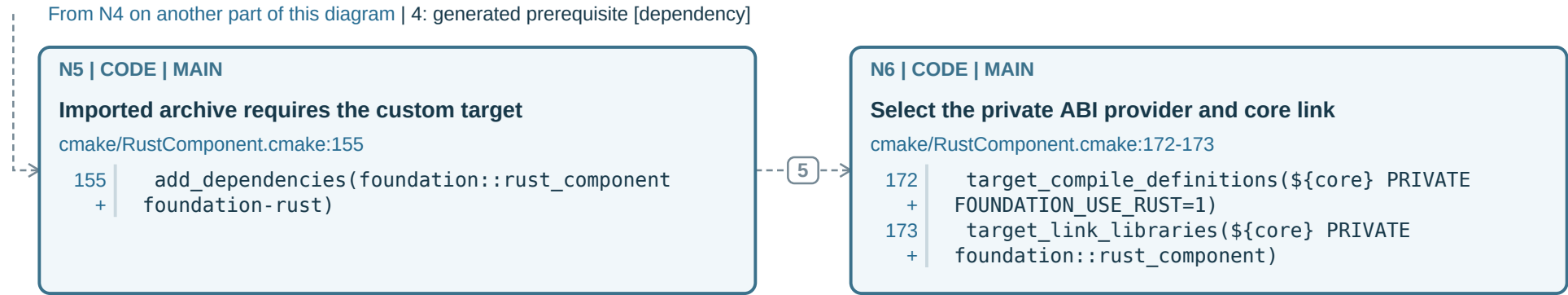


1. N1 -> N2 [conditional] if provider = rust; after omitted checks
2. N2 -> N3 [process] per-configuration description
3. N3 -> N4 [step] later declarations use described paths

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: compile | Evidence and other parents | Interactive diagram

Diagram part 2 of 2 | Previous part

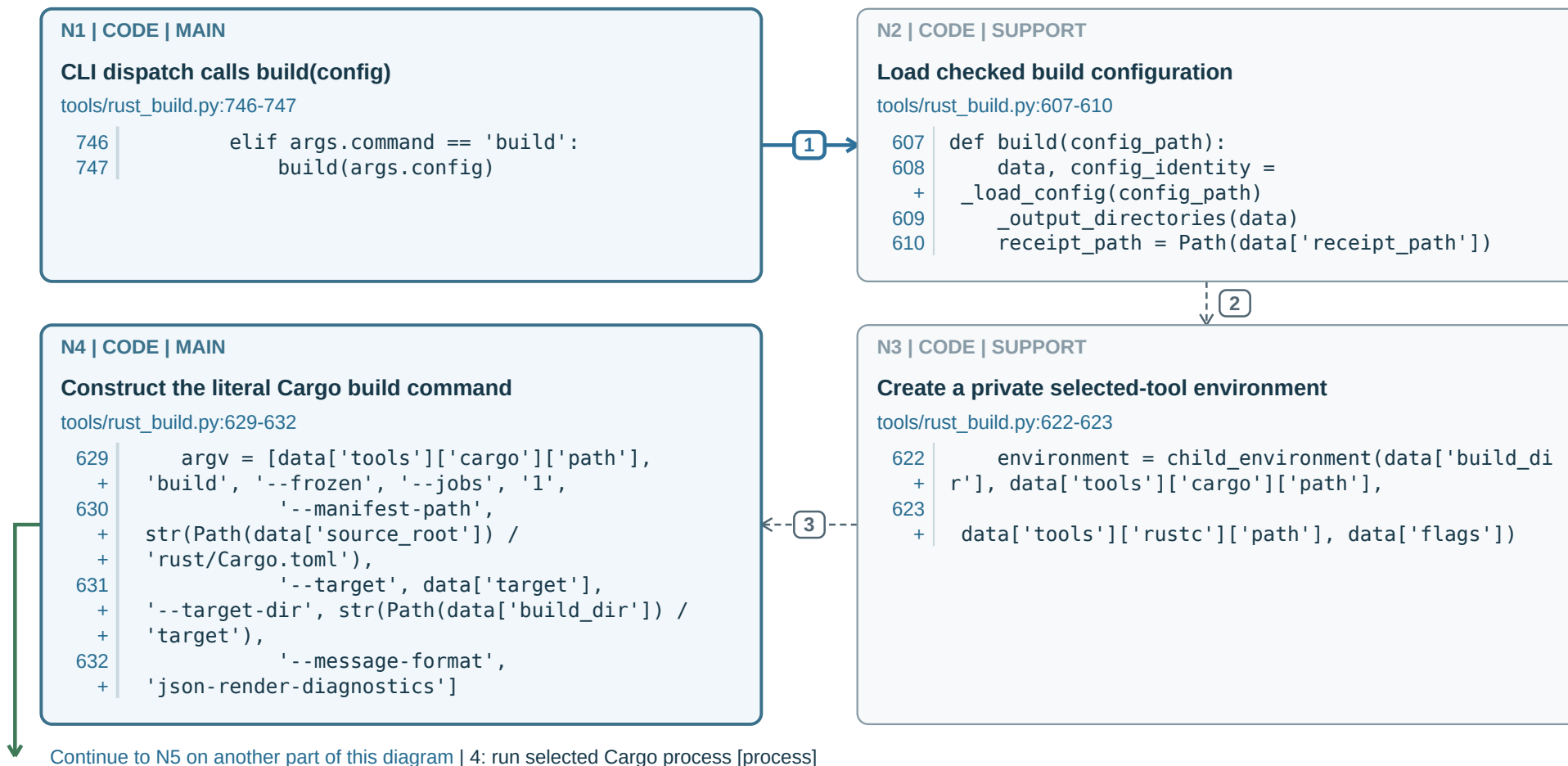


5. N5 -> N6 [dependency] selected static component link

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [rust-component / custom-target](#) | [Evidence and other parents](#) | [Interactive diagram](#)

Diagram part 1 of 2 | [Next part](#)



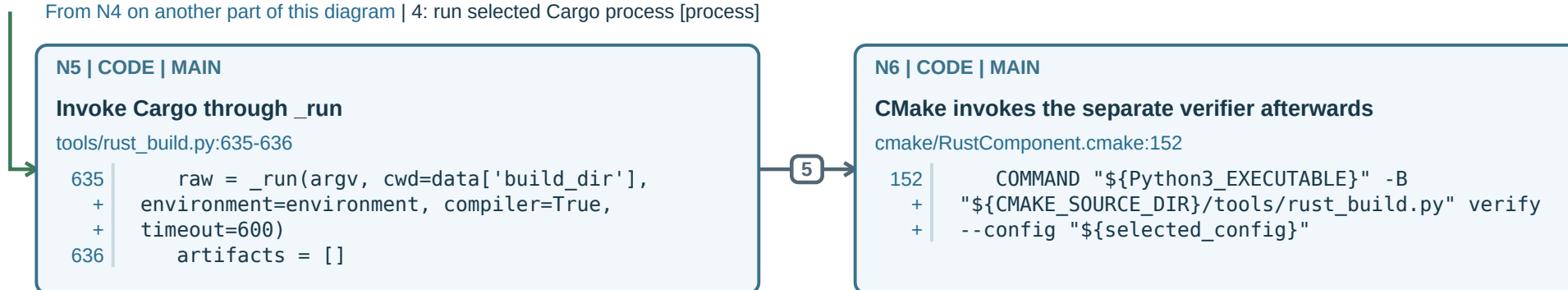
1. N1 -> N2 [call] build(args.config)
2. N2 -> N3 [step] after prior receipt checks
3. N3 -> N4 [step] after cache invalidation if needed

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

[Up: rust-component / custom-target](#) | [Evidence and other parents](#) | [Interactive diagram](#)

[Diagram part 2 of 2](#) | [Previous part](#)

[From N4 on another part of this diagram](#) | 4: run selected Cargo process [process]

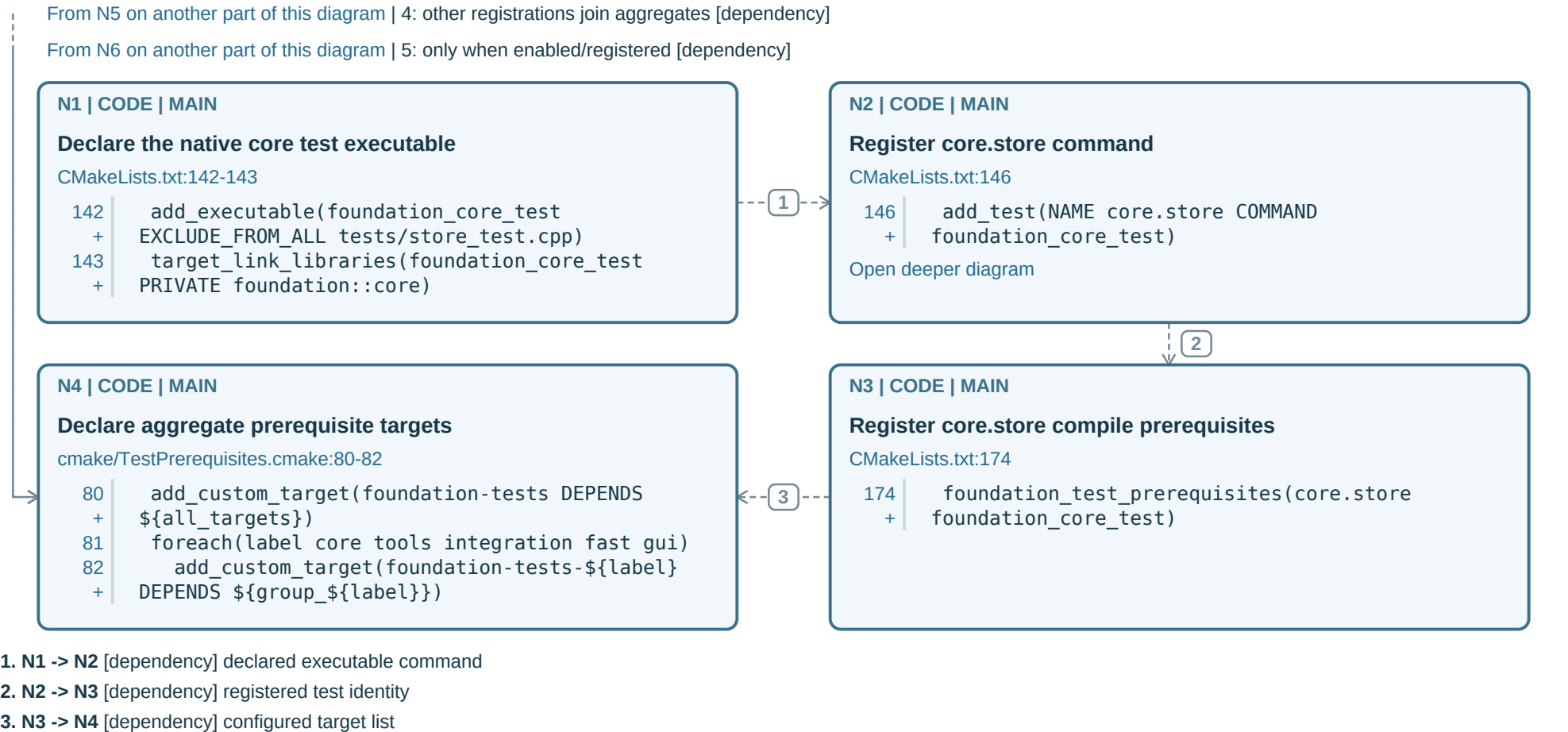


5. N5 -> N6 [step] custom target next command; intervening receipt checks

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: test | Evidence and other parents | Interactive diagram

Diagram part 1 of 2 | Next part



MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [test](#) | [Evidence and other parents](#) | [Interactive diagram](#)

Diagram part 2 of 2 | [Previous part](#)

N5 | CODE | MAIN

Register a tools.* Python suite command

CMakeLists.txt:239-240

```
239 |         add_test(NAME tools.${suite} COMMAND
+ |         ${Python3_EXECUTABLE} "${CMAKE_CURRENT_SOURCE_DIR}
+ |         }/tools/run_tests.py"
240 |         --suite "${suite}" --output
+ |         "${CMAKE_BINARY_DIR}/test-reports/${suite}.json")
```

[Open deeper diagram](#)

N6 | CODE | SUPPORT

Register exact-host native Rust unit tests

cmake/RustComponent.cmake:156-158

```
156 |         if(BUILD_TESTING AND NOT EMSCRIPTEN AND
+ |         compiler_host STREQUAL target)
157 |             add_test(NAME rust.unit COMMAND
+ |             "${Python3_EXECUTABLE}" -B
+ |             "${CMAKE_SOURCE_DIR}/tools/rust_build.py"
158 |             test --config "${selected_config}")
```

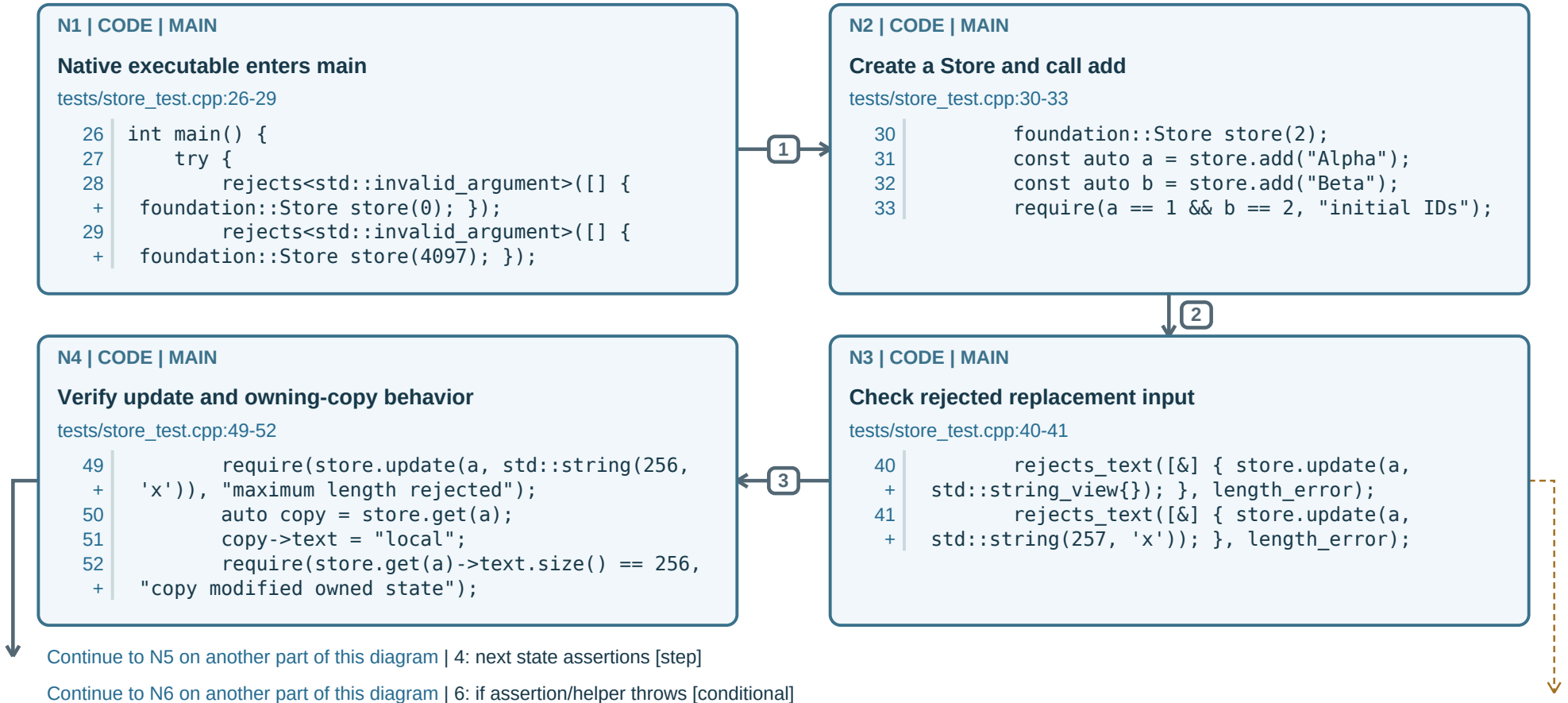
[Continue to N4 on another part of this diagram](#) | 4: other registrations join aggregates [dependency]

[Continue to N4 on another part of this diagram](#) | 5: only when enabled/registered [dependency]

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: test | Evidence and other parents | Interactive diagram

Diagram part 1 of 2 | Next part



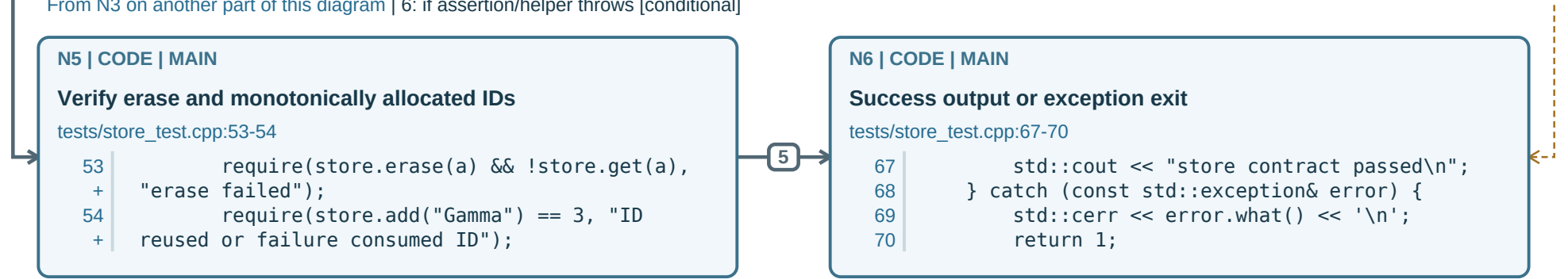
- 1. N1 -> N2 [step] after invalid-capacity assertions
- 2. N2 -> N3 [step] after capacity/text checks
- 3. N3 -> N4 [step] after remaining rejection assertions

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [test](#) | [Evidence and other parents](#) | [Interactive diagram](#)

Diagram part 2 of 2 | [Previous part](#)

From N4 on another part of this diagram | 4: next state assertions [step]
From N3 on another part of this diagram | 6: if assertion/helper throws [conditional]

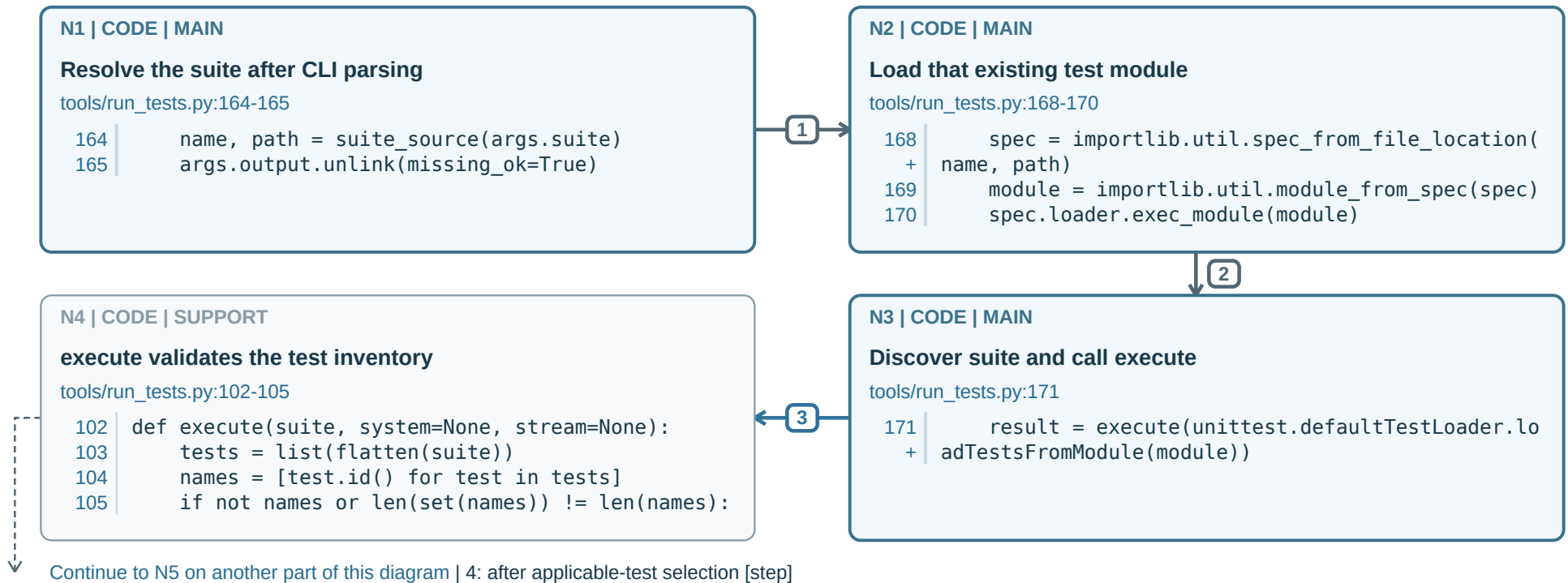


5. N5 -> N6 [step] after all-byte loop

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [test](#) | [Evidence and other parents](#) | [Interactive diagram](#)

Diagram part 1 of 2 | [Next part](#)



1. N1 -> N2 [step] selected module path

2. N2 -> N3 [step] loaded module

3. N3 -> N4 [call] execute(suite)

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [test](#) | [Evidence and other parents](#) | [Interactive diagram](#)

Diagram part 2 of 2 | [Previous part](#)

From N4 on another part of this diagram | 4: after applicable-test selection [step]

N5 | CODE | MAIN

Run applicable tests and require complete outcomes

tools/run_tests.py:117-120

```
117     result = unittest.TextTestRunner(stream=stream
+   or sys.stderr, verbosity=2,
+   resultclass=Recorded).run(unittest.TestSuite(selected))
118     required = {test.id() for test in selected}
119     complete = (result.wasSuccessful() and
+   set(result.outcomes) == required and
120               all(item['status'] == 'passed' for
+   item in result.outcomes.values()))
```

5

N6 | CODE | MAIN

Back in main: publish result and return status

tools/run_tests.py:172-173

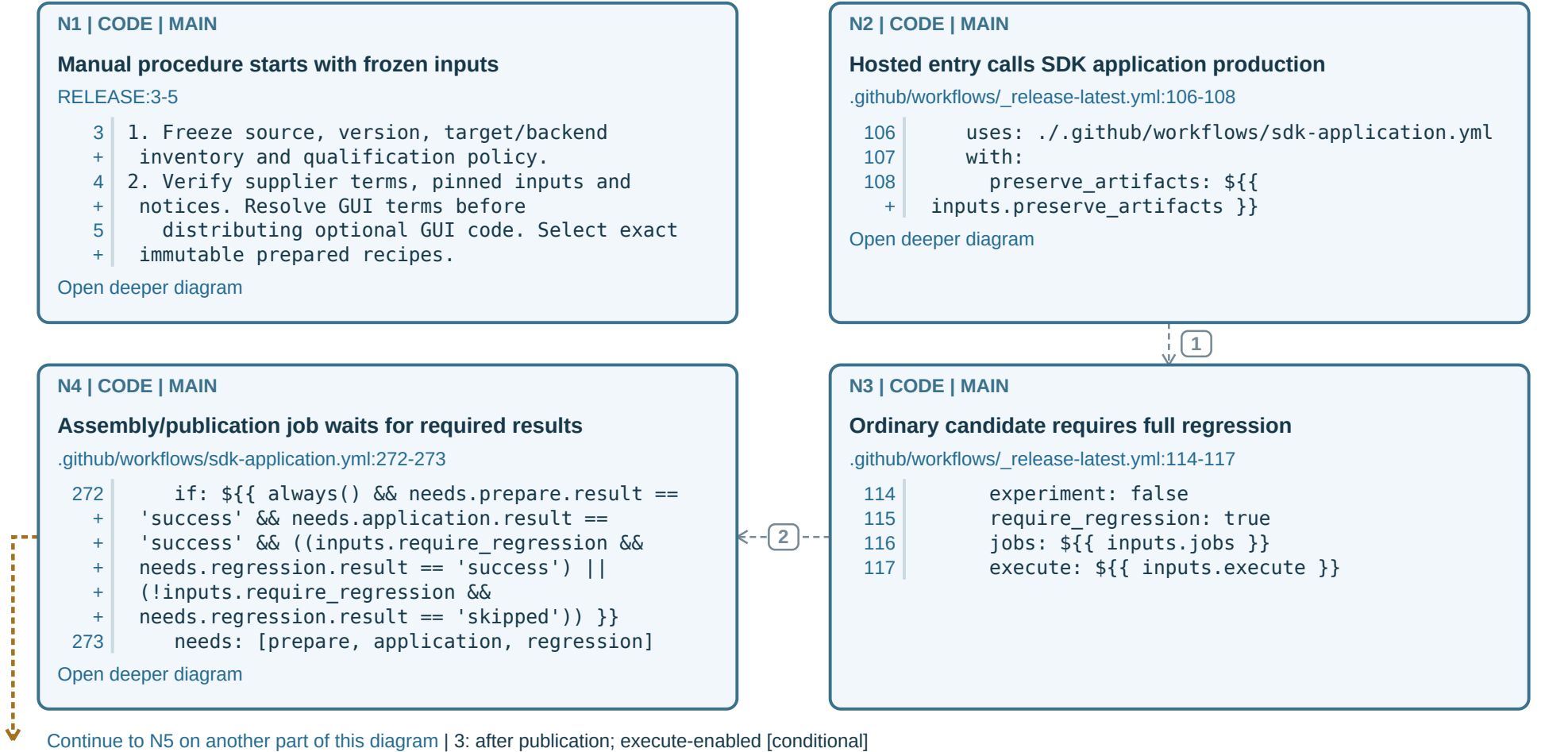
```
172     publish(args.output, result)
173     return 0 if result['status'] == 'passed' else
+   1
```

5. N5 -> N6 [return] execute returns result to main

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: release | Evidence and other parents | Interactive diagram

Diagram part 1 of 2 | Next part

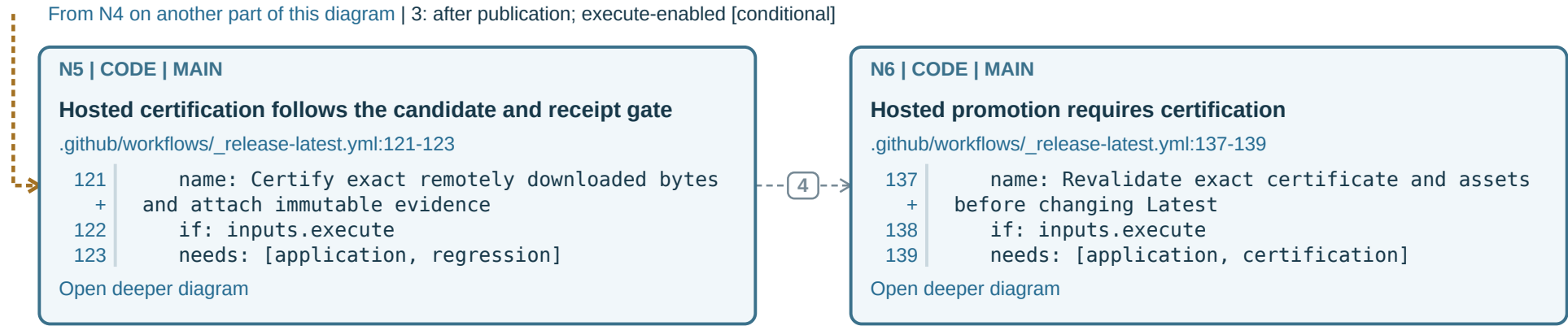


- 1. N2 -> N3 [dependency] workflow inputs
- 2. N3 -> N4 [dependency] required successful regression

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [release](#) | [Evidence and other parents](#) | [Interactive diagram](#)

Diagram part 2 of 2 | [Previous part](#)

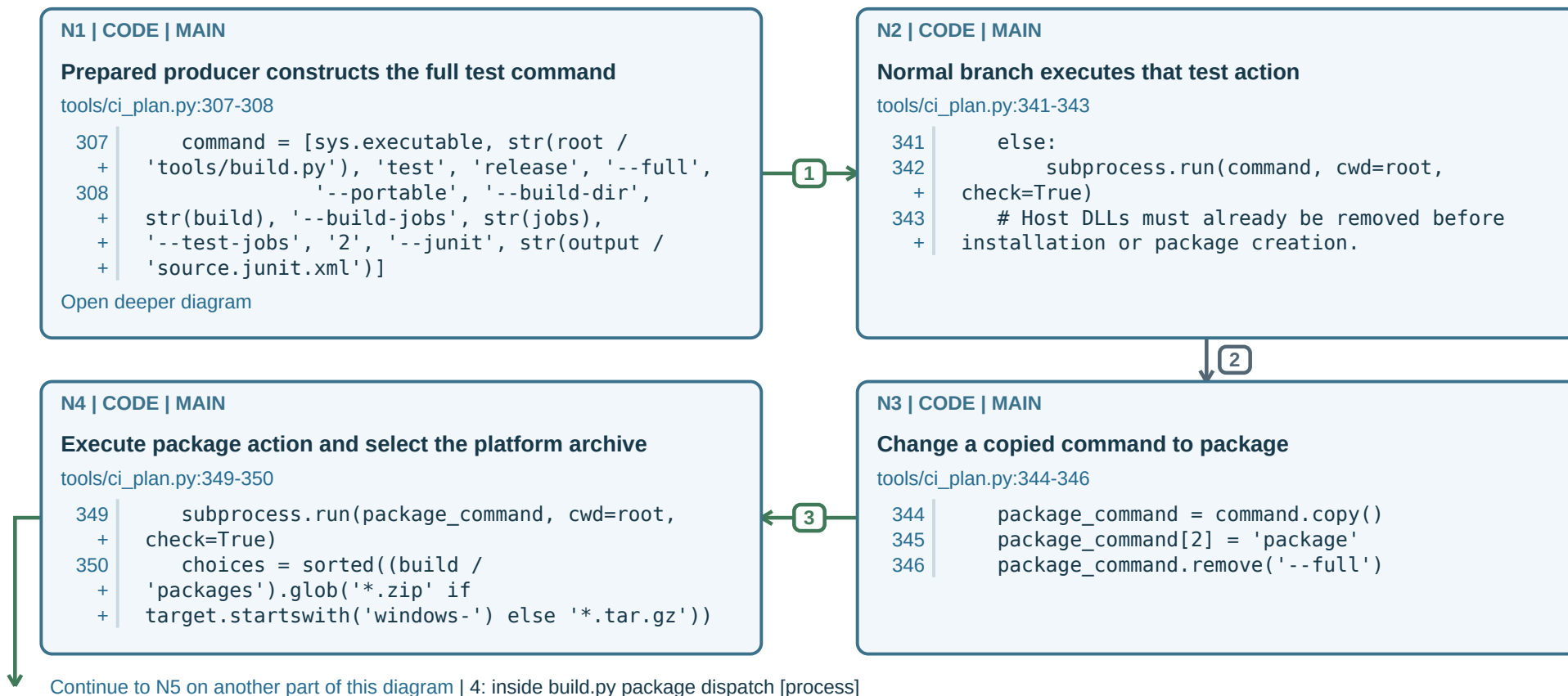


4. N5 -> N6 [dependency] successful certificate handoff

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [release](#) | [Evidence and other parents](#) | [Interactive diagram](#)

Diagram part 1 of 2 | [Next part](#)



1. N1 -> N2 [process] after selected input flags

2. N2 -> N3 [step] test returned successfully

3. N3 -> N4 [process] separate package process

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

[Up: release](#) | [Evidence and other parents](#) | [Interactive diagram](#)

[Diagram part 2 of 2](#) | [Previous part](#)

[From N4 on another part of this diagram](#) | 4: inside build.py package dispatch [process]

N5 | CODE | MAIN

Inside build.execute: launch CPack

tools/build.py:501

```
501 |         timings.call("package", run,  
+   |         [programs["cpack"], "--config", str(build /  
+   |         "CPackConfig.cmake"), "-C", "Release"],  
+   |         env=child_environment)
```

..... 5

N6 | CODE | MAIN

Back in producer: describe and verify copied archive

tools/ci_plan.py:357-359

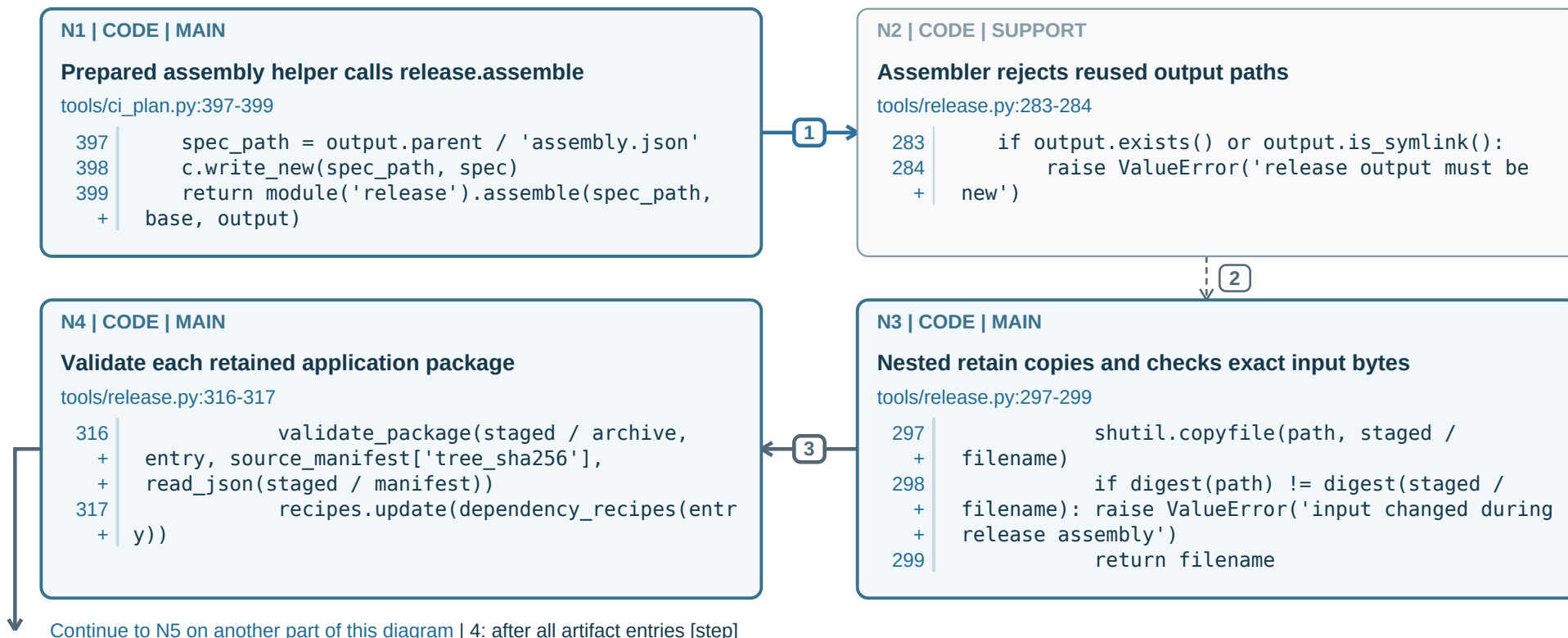
```
357 |         module('coverage').write_new(descriptor,  
+   |         a.describe(archive))  
358 |         a.verify(archive, descriptor, sdk=work / 'sdk'  
+   |         if (work / 'sdk/sdk.json').is_file() else None,  
359 |                 abi=target.startswith('linux-'),  
+   |                 processor=target.split('- ', 1)[1])
```

5. N5 -> N6 [return] package returns; copy selected archive first

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [release](#) | [Evidence and other parents](#) | [Interactive diagram](#)

Diagram part 1 of 2 | [Next part](#)



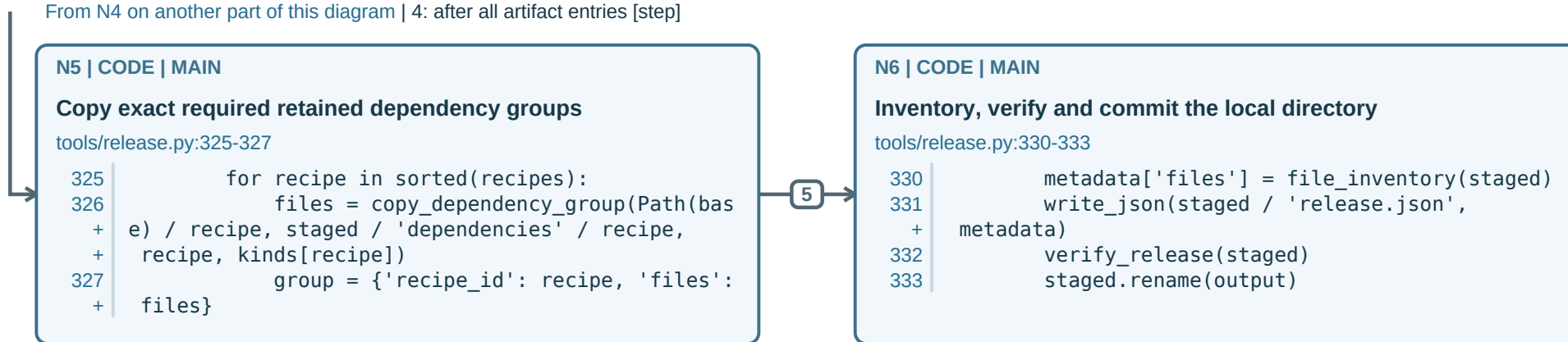
1. N1 -> N2 [call] assemble(spec_path, base, output)
2. N2 -> N3 [step] after new temporary staging
3. N3 -> N4 [step] source verified; process each artifact

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

[Up: release](#) | [Evidence and other parents](#) | [Interactive diagram](#)

[Diagram part 2 of 2](#) | [Previous part](#)

[From N4 on another part of this diagram](#) | 4: after all artifact entries [step]

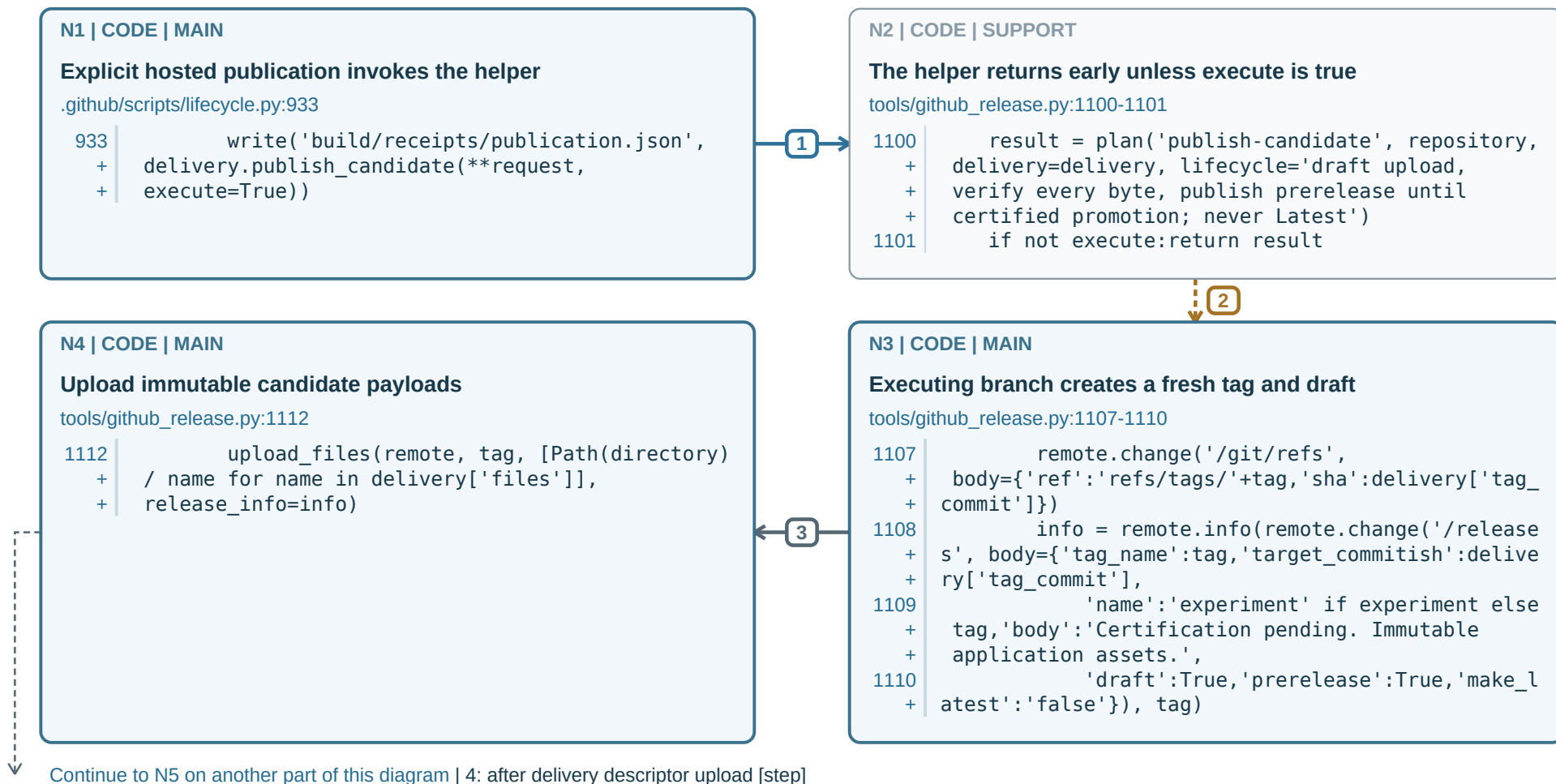


5. N5 -> N6 [step] complete local inputs

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [release](#) | [Evidence and other parents](#) | [Interactive diagram](#)

Diagram part 1 of 2 | [Next part](#)



1. N1 -> N2 [call] publish_candidate(**request)
2. N2 -> N3 [conditional] only if execute=True; identity checks omitted
3. N3 -> N4 [step] after observing draft

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

[Up: release](#) | [Evidence and other parents](#) | [Interactive diagram](#)

[Diagram part 2 of 2](#) | [Previous part](#)

[From N4 on another part of this diagram](#) | 4: after delivery descriptor upload [step]

N5 | CODE | SUPPORT

Verify the uploaded draft identity and bytes

tools/github_release.py:1115-1116

```
1115         current, assets = verified_remote(remote,
+         delivery, directory, draft=True, prerelease=True,
+         metadata_only=True)
1116         if current['id'] != info['id']:raise
+         DeliveryError('draft release identity changed')
```

5

N6 | CODE | MAIN

Publish prerelease and ensure it is outside Latest

tools/github_release.py:1117-1120

```
1117         remote.change(f'/releases/{info["id"]}',
+         method='PATCH', body={'draft':False,'prerelease':
+         True,'make_latest':'false'})
1118         final, final_assets =
+         verified_remote(remote, delivery, directory,
+         prerelease=True, readback=False,
+         metadata_only=True)
1119         if final['id'] != info['id'] or
+         final_assets != assets:raise
+         DeliveryError('publication identity changed')
1120         remote.not_latest(final)
```

[Open deeper diagram](#)

5. N5 -> N6 [step] verified draft identity

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [release](#) | [Evidence and other parents](#) | [Interactive diagram](#)

Diagram part 1 of 2 | [Next part](#)

N1 | CODE | MAIN

Dispatcher invokes prepare or run for a physical check

[.github/scripts/lifecycle.py:955-958](#)

```
955         operation = qualification_tasks.prepare if
+   command == 'check-prerequisites' else
+   qualification_tasks.run
956         result = operation(ROOT, value('CHECK'),
+   value('GITHUB_RUN_ID'), int(value('GITHUB_RUN_ATT
+   EMPT'))))
957         if command == 'check' and result['status']
+   != 'passed':
958             raise ValueError('required
+   qualification did not pass')
```

1

N2 | CODE | MAIN

Certificate dispatcher calls certify

[.github/scripts/lifecycle.py:966-968](#)

```
966         result = certify_release.certify(direc
+   tory, ci.module('release').verify_metadata(directo
+   ry), plan, reports,
967         evidence.load(policy),
+   value('PROFILE'), identity['experiment'],
968         adoption=evidence.load(ROOT
+   / 'build/adoption.json') if (ROOT /
+   'build/adoption.json').exists() else None)
```

2

N4 | CODE | MAIN

Returned certificate marks promotion eligibility

[tools/certify_release.py:99](#)

```
99         "eligible_for_promotion": passed and
+   not experiment, "experiment": experiment,
```

3

N3 | CODE | SUPPORT

certify merges reports and checks actual host evidence

[tools/certify_release.py:90-93](#)

```
90         result = coverage.merge(plan, reports,
+   adoption=adoption)
91         report_paths = {coverage.load(path)["check"]:
+   path for path in reports}
92         for check in plan["checks"]:
93             coverage.check_host(plan, check['id'],
+   result['checks'][check['id']],
+   report_paths[check['id']].parent)
```

Continue to N5 on another part of this diagram | 4: separate execute-enabled attachment operation [step]

1. N1 -> N2 [step] reports handed to certificate job
2. N2 -> N3 [call] certify(...); after subject guards
3. N3 -> N4 [step] compute and return certificate

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [release](#) | [Evidence and other parents](#) | [Interactive diagram](#)

Diagram part 2 of 2 | [Previous part](#)

From N4 on another part of this diagram | 4: [separate execute-enabled attachment operation](#) [step]

N5 | CODE | MAIN

A separate dispatcher operation executes attachment

[.github/scripts/lifecycle.py:978-981](#)

```
978         write('build/receipts/attachment.json'
+         , delivery.attach_certificate(value('GITHUB_REPOSI
+         TORY'), value('TAG'), directory,
979             identity,
+         Path('build/certificate.json'),
+         Path('build/check-plan.json'), policy,
+         value('PROFILE'),
980             reports,
+         ci_retry.origin_attempt(value('CERTIFICATE_ATTEMP
+         T'), int(value('GITHUB_RUN_ATTEMPT'))),
981             execute=True,
+         metadata_only=True))
```

5

N6 | CODE | SUPPORT

Attachment helper defaults to planning

[tools/github_release.py:1200-1201](#)

```
1200 def attach_certificate(repository, tag, directory,
+         delivery, certificate, check_plan, policy,
+         profile,
1201             reports, attempt, *,
+         execute=False, transport=None,
+         metadata_only=False):
```

[Open deeper diagram](#)

5. N5 -> N6 [call] `attach_certificate(...)`

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [release](#) | [Evidence and other parents](#) | [Interactive diagram](#)Diagram part 1 of 2 | [Next part](#)

N1 | CODE | MAIN

Promotion dispatcher explicitly selects plan or execution

[.github/scripts/lifecycle.py:991](#)

```

991 |         write('build/receipts/' + command +
+   |         '.json', delivery.promote(**request,
+   |         execute=command == 'promote'))

```

1

N2 | CODE | SUPPORT

Promote rejects experiment/base/mismatched subjects

[tools/github_release.py:1281-1284](#)

```

1281 | def promote(repository, tag, directory, delivery, poli
+   | cy, profile, run_id, attempt, certificate_sha256, *, exe
+   | cute=False, transport=None, metadata_only=False):
1282 |     location(repository, tag); validate_delivery(del
+   | ivery, directory, metadata_only=metadata_only)
1283 |     if (repository, tag) != (delivery['repository'], d
+   | elivery['tag']) or delivery['experiment'] or
+   | tag=='base':
1284 |         raise DeliveryError('only this ordinary
+   | application release can be promoted')

```

2

N4 | CODE | MAIN

Change the exact release to stable Latest

[tools/github_release.py:1300](#)

```

1300 |         remote.change(f'/releases/{info["id"]}', me
+   | thod='PATCH', body={'draft': False, 'prerelease': Fals
+   | e, 'make_latest': 'true'})

```

3

N3 | CODE | SUPPORT

Executing branch revalidates assets and certificate

[tools/github_release.py:1294-1295](#)

```

1294 |         remote.visible(); info, assets=verified_remo
+   | te(remote, delivery, directory, readback=not
+   | metadata_only, metadata_only=metadata_only)
1295 |         evidence=verify_certificate(remote, assets,
+   | delivery, directory, policy, profile, run_id, attempt, c
+   | ertificate_sha256, metadata_only=metadata_only)

```

Continue to N5 on another part of this diagram | 4: after lifecycle/assets/tag recheck [step]

1. N1 -> N2 [call] promote(**request)
2. N2 -> N3 [conditional] only execute=True; more guards omitted
3. N3 -> N4 [step] exact evidence + unchanged identity/policy

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

[Up: release](#) | [Evidence and other parents](#) | [Interactive diagram](#)

[Diagram part 2 of 2](#) | [Previous part](#)

[From N4 on another part of this diagram](#) | 4: after lifecycle/assets/tag recheck [step]

N5 | CODE | MAIN

Read back and verify the Latest pointer

tools/github_release.py:1304-1307

```
1304         latest=remote.transport.json(remote.base+'
+ /releases/latest')
1305         remote.info(latest,tag)
1306         if latest['id']!=info['id'] or
+ latest['draft'] or latest['prerelease']:
1307             raise DeliveryError('Latest pointer
+ does not identify the exact certified release')
```

---(5)--->

N6 | CODE | SUPPORT

Hosted final job independently verifies the outcome

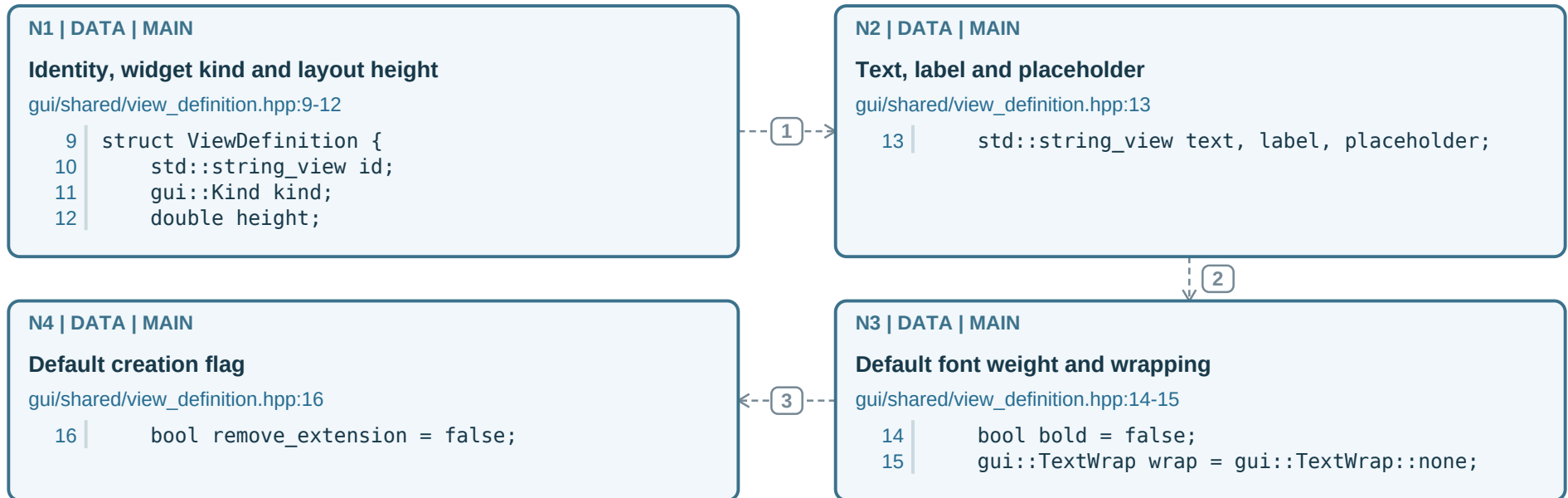
.github/workflows/_release-latest.yml:167

```
167 | - run: python3 -B tools/latest_release.py
+ | verify --output build/latest-verification.json
```

5. N5 -> N6 [step] separate final workflow job

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [widget-definitions / fields](#) | [Evidence and other parents](#) | [Interactive diagram](#)

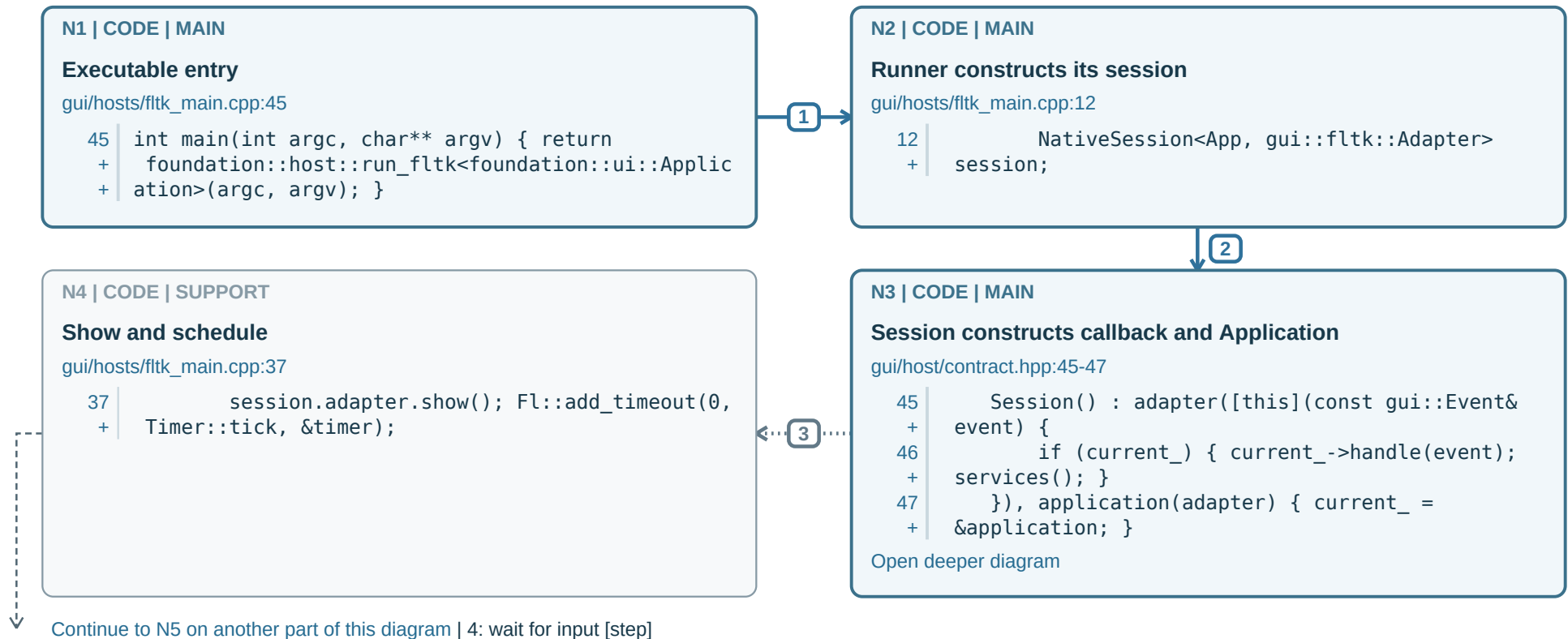


1. N1 -> N2 [dependency] following aggregate members
2. N2 -> N3 [dependency] following aggregate members / defaults
3. N3 -> N4 [dependency] final aggregate member / default

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: startup | Evidence and other parents | Interactive diagram

Diagram part 1 of 2 | Next part

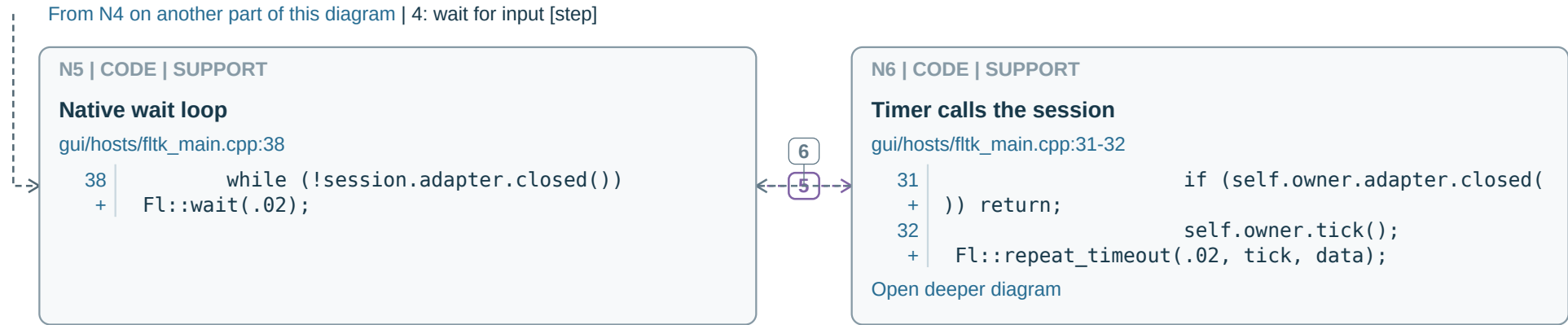


1. N1 -> N2 [call] run_ftk; argument checks passed
2. N2 -> N3 [call] NativeSession aliases Session
3. N3 -> N4 [return] construction/publish returned; normal branch

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [startup](#) | [Evidence and other parents](#) | [Interactive diagram](#)

Diagram part 2 of 2 | [Previous part](#)



5. N5 -> N6 [event] scheduled callback

6. N6 -> N5 [return] return / reschedule

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [main-to-widgets / session](#) | [Evidence and other parents](#) | [Interactive diagram](#)

Diagram part 1 of 2 | [Next part](#)

N1 | CODE | SUPPORT

NativeSession is a typed Session alias

gui/host/native_session.hpp:7

7 | template<Application App, class Adapter> using
+ | NativeSession = Session<App, Adapter,
+ | FileServices>;

N2 | CODE | SUPPORT

Session contains adapter and application

gui/host/contract.hpp:43-44

43 | Adapter adapter;
44 | App application;

N4 | CODE | MAIN

Application borrows adapter and owns view

gui/shared/application.hpp:38-39

38 | gui::Adapter& adapter_
39 | gui::Snapshot view_;

N3 | CODE | MAIN

Session supplies adapter to Application

gui/host/contract.hpp:45-47

45 | Session() : adapter([this](const gui::Event&
+ | event) {
46 | if (current_) { current_->handle(event);
+ | services(); }
47 | }), application(adapter) { current_ =
+ | &application; }
Open deeper diagram



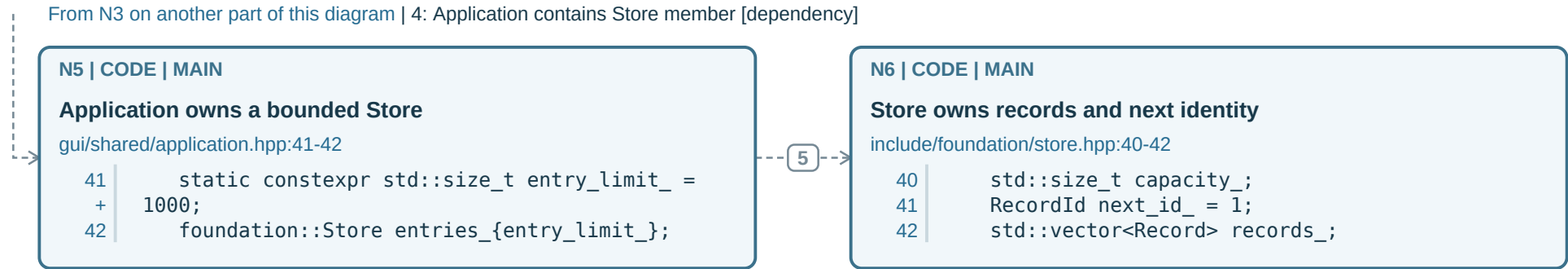
Continue to N5 on another part of this diagram | 4: Application contains Store member [dependency]

- 1. N1 -> N2 [dependency] typed alias instantiates Session
- 2. N2 -> N3 [dependency] members initialized by this constructor
- 3. N3 -> N4 [dependency] adapter passed by reference

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [main-to-widgets / session](#) | [Evidence and other parents](#) | [Interactive diagram](#)

Diagram part 2 of 2 | [Previous part](#)

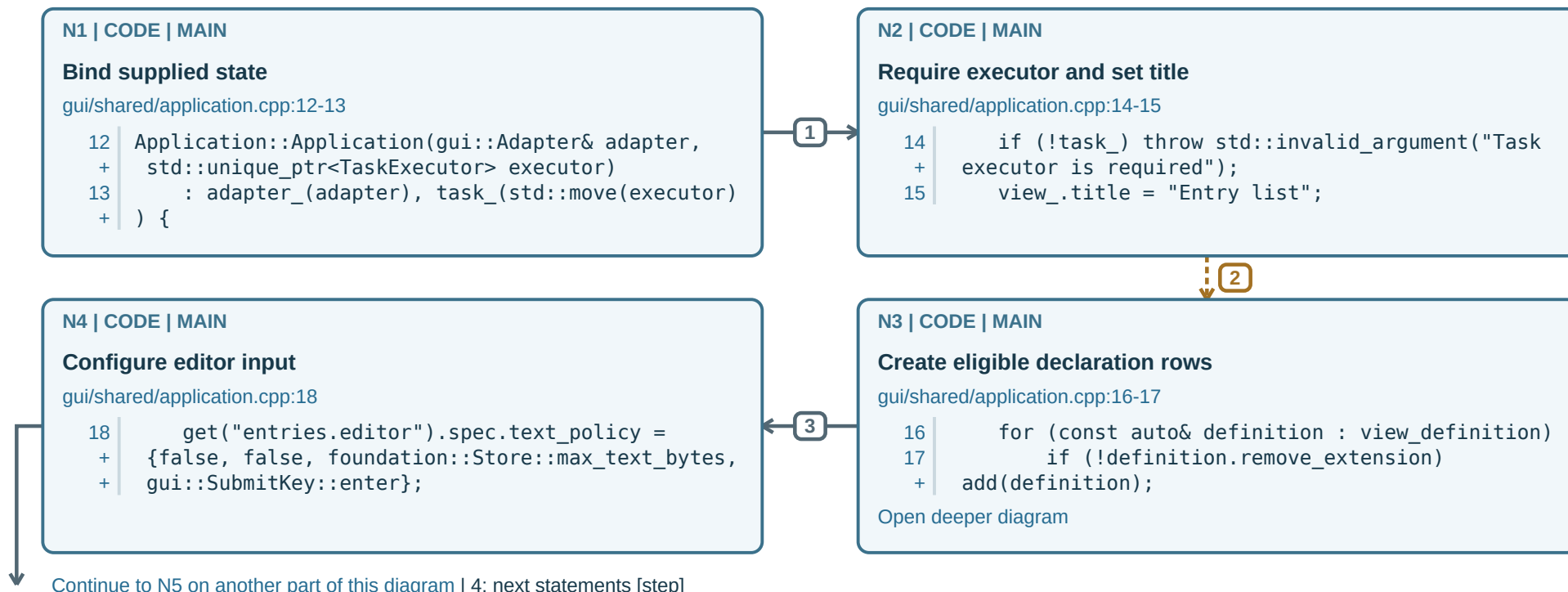


5. N5 -> N6 [dependency] Store owns value records / monotonic ID

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [startup](#) | [Evidence and other parents](#) | [Interactive diagram](#)

Diagram part 1 of 2 | [Next part](#)



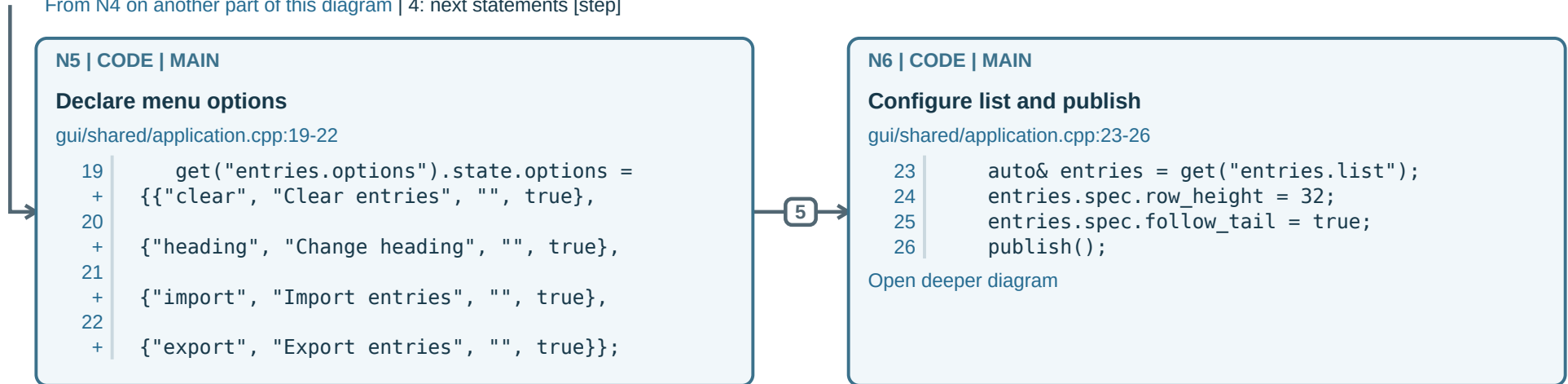
1. N1 -> N2 [step] constructor body
2. N2 -> N3 [conditional] executor present
3. N3 -> N4 [step] creation loop completed

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

[Up: startup](#) | [Evidence and other parents](#) | [Interactive diagram](#)

[Diagram part 2 of 2](#) | [Previous part](#)

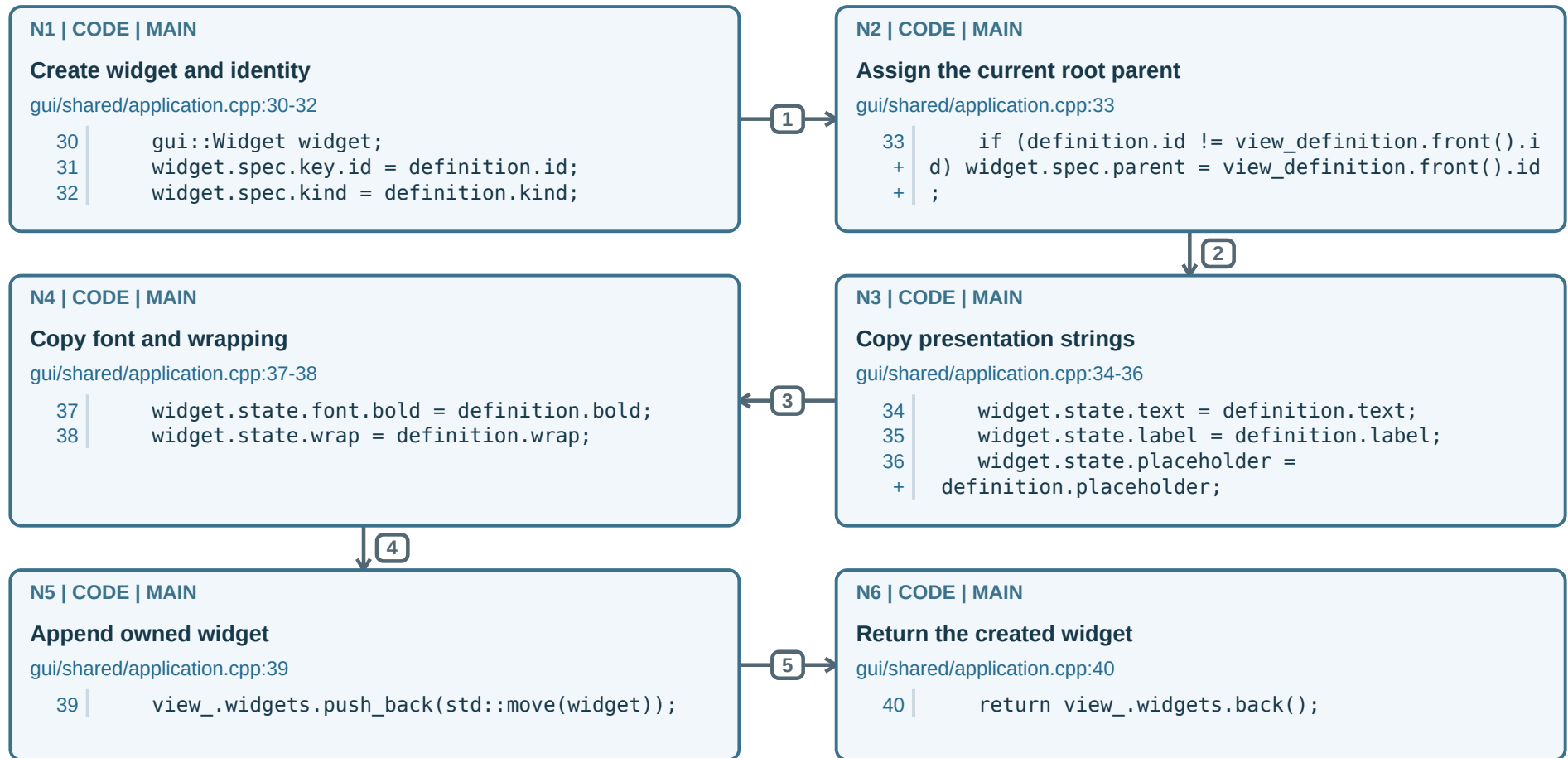
[From N4 on another part of this diagram](#) | 4: next statements [step]



5. N5 -> N6 [step] list setup; next publish call

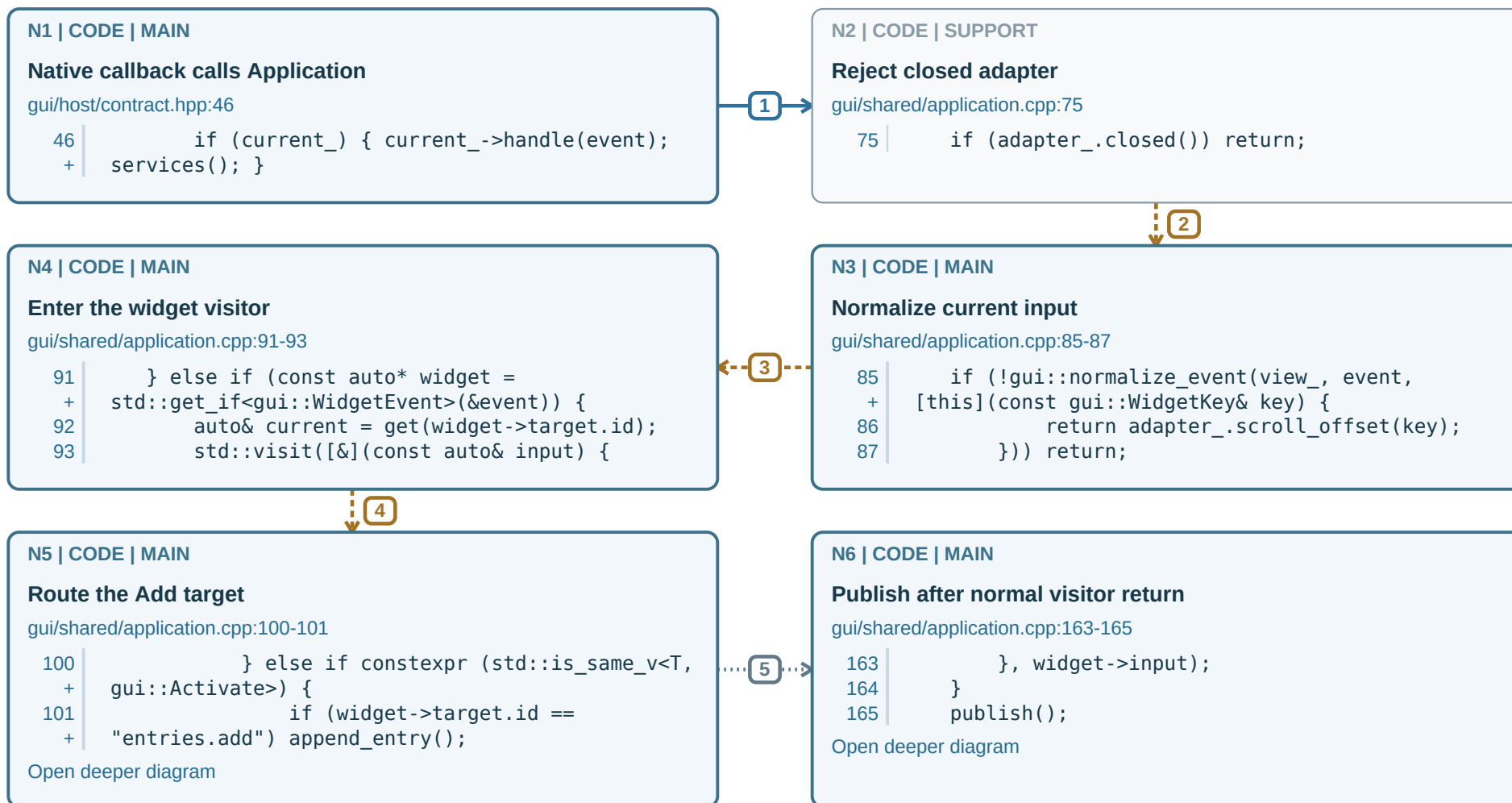
MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [main-to-widgets](#) / [add](#) | [Evidence and other parents](#) | [Interactive diagram](#)



1. N1 -> N2 [step] next statement
2. N2 -> N3 [step] after condition; root skips assignment
3. N3 -> N4 [step] next statements
4. N4 -> N5 [step] append
5. N5 -> N6 [step] insertion succeeded

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [button-click](#) | [Evidence and other parents](#) | [Interactive diagram](#)

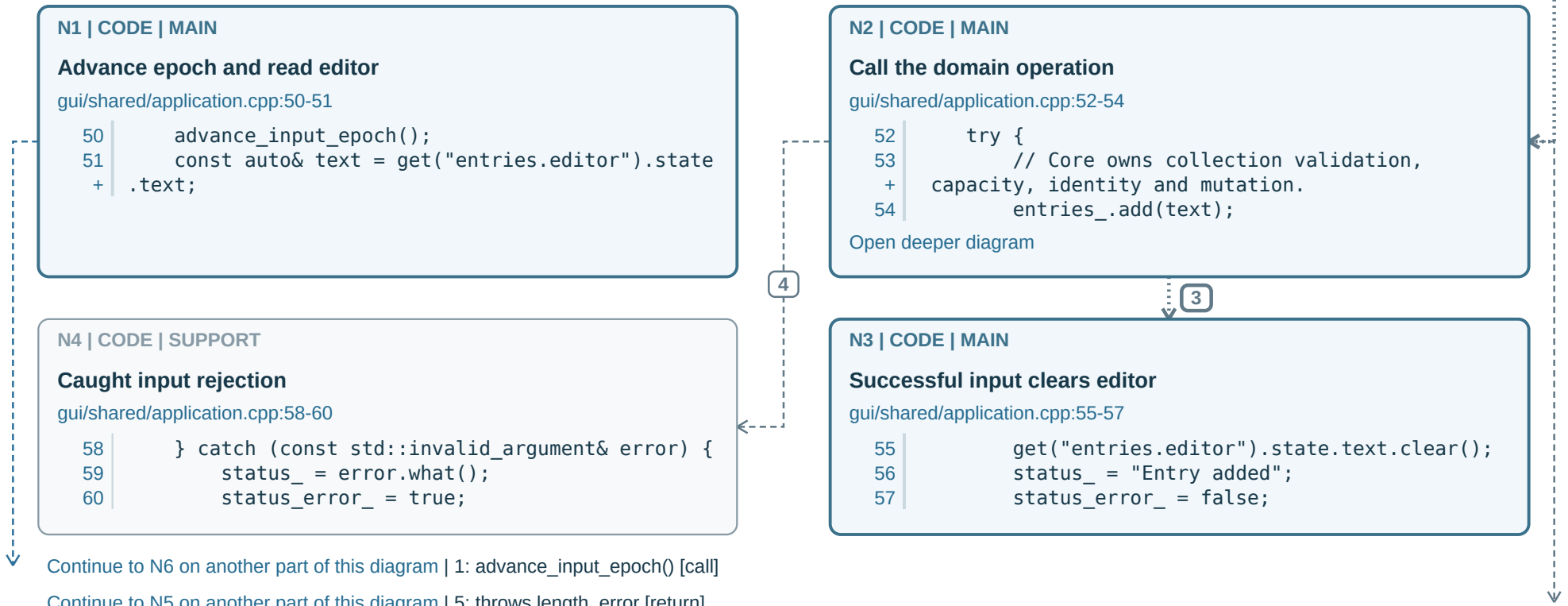
1. N1 -> N2 [call] handle(event)
2. N2 -> N3 [conditional] open; close/shutdown checks passed
3. N3 -> N4 [conditional] accepted WidgetEvent
4. N4 -> N5 [conditional] visitor selects Activate; Add ID
5. N5 -> N6 [return] append_entry and visitor return normally

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: button-click | Evidence and other parents | Interactive diagram

Diagram part 1 of 2 | Next part

From N6 on another part of this diagram | 2: epoch advanced; editor read [return]



- 3. N2 -> N3 [return] Store::add returns
- 4. N2 -> N4 [return] throws invalid_argument

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [button-click](#) | [Evidence and other parents](#) | [Interactive diagram](#)

Diagram part 2 of 2 | [Previous part](#)

From N1 on another part of this diagram | 1: `advance_input_epoch()` [call]

From N2 on another part of this diagram | 5: `throws length_error` [return]

N5 | [CODE](#) | [SUPPORT](#)

Caught capacity failure

`gui/shared/application.cpp:61-63`

```
61         } catch (const std::length_error& error) {
62             status_ = error.what();
63             status_error_ = true;
```

Continue to N2 on another part of this diagram | 2: `epoch advanced; editor read` [return]

N6 | [CODE](#) | [SUPPORT](#)

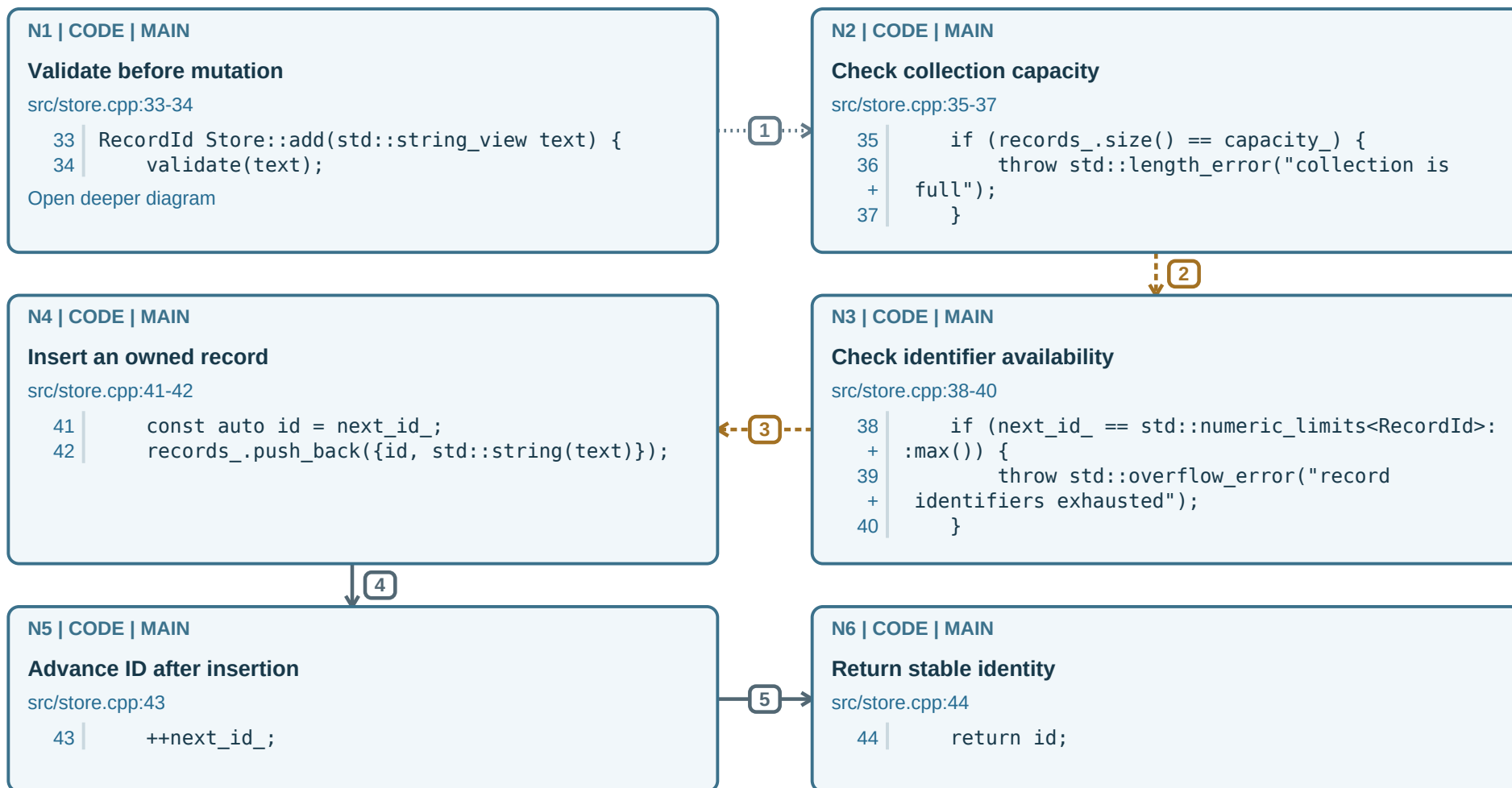
Epoch helper rejects exhaustion first

`gui/shared/application.cpp:68-71`

```
68         if (input_epoch_ == std::numeric_limits<std::u
+ int64_t>::max()) {
69             throw std::overflow_error("Input epoch
+ exhausted");
70         }
71         ++input_epoch_;
```

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

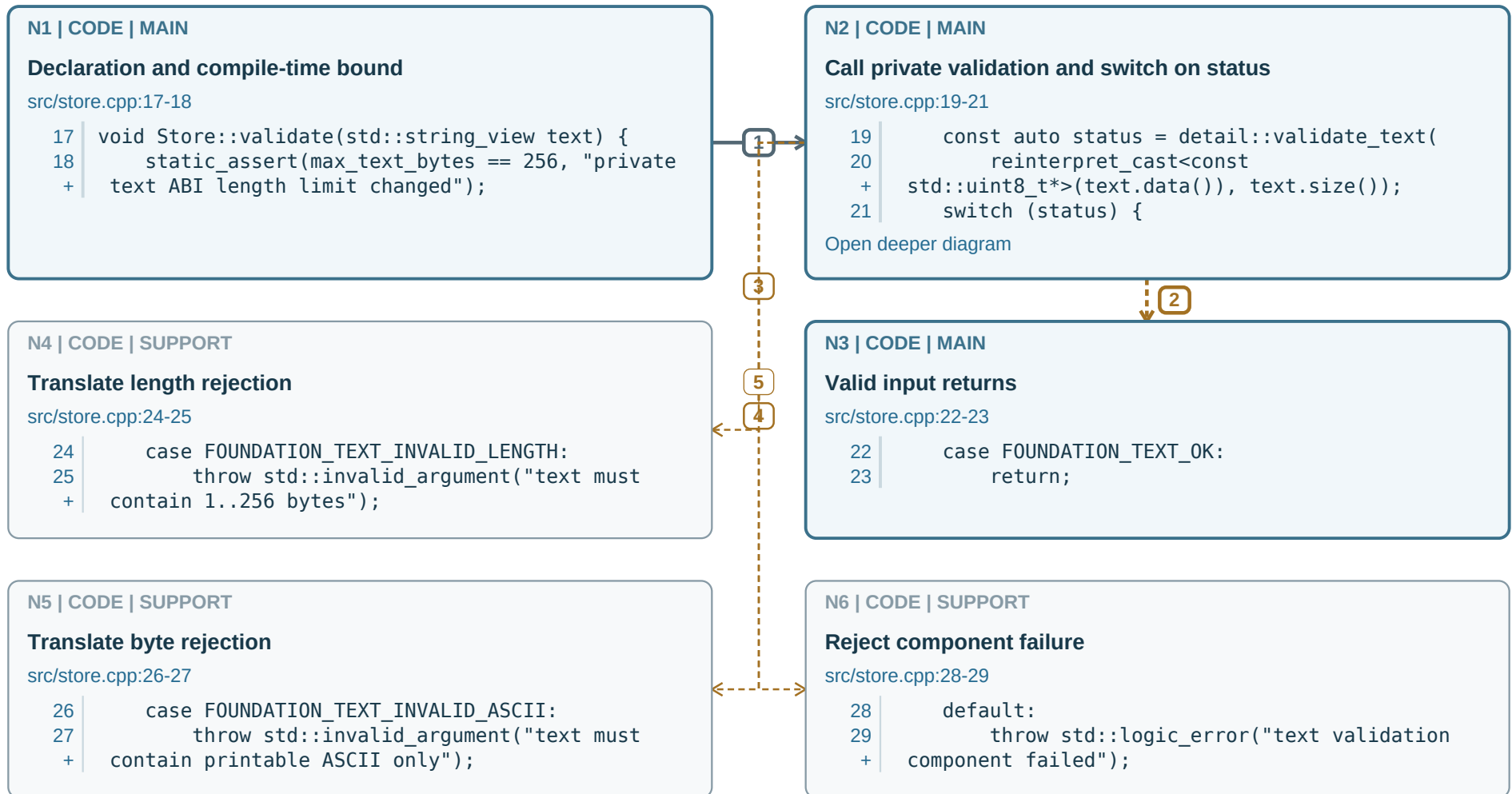
Up: [button-click](#) | [Evidence and other parents](#) | [Interactive diagram](#)



1. N1 -> N2 [return] validation returns
2. N2 -> N3 [conditional] not full
3. N3 -> N4 [conditional] ID available
4. N4 -> N5 [step] insertion succeeded
5. N5 -> N6 [step] return to caller

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [button-click](#) | [Evidence and other parents](#) | [Interactive diagram](#)

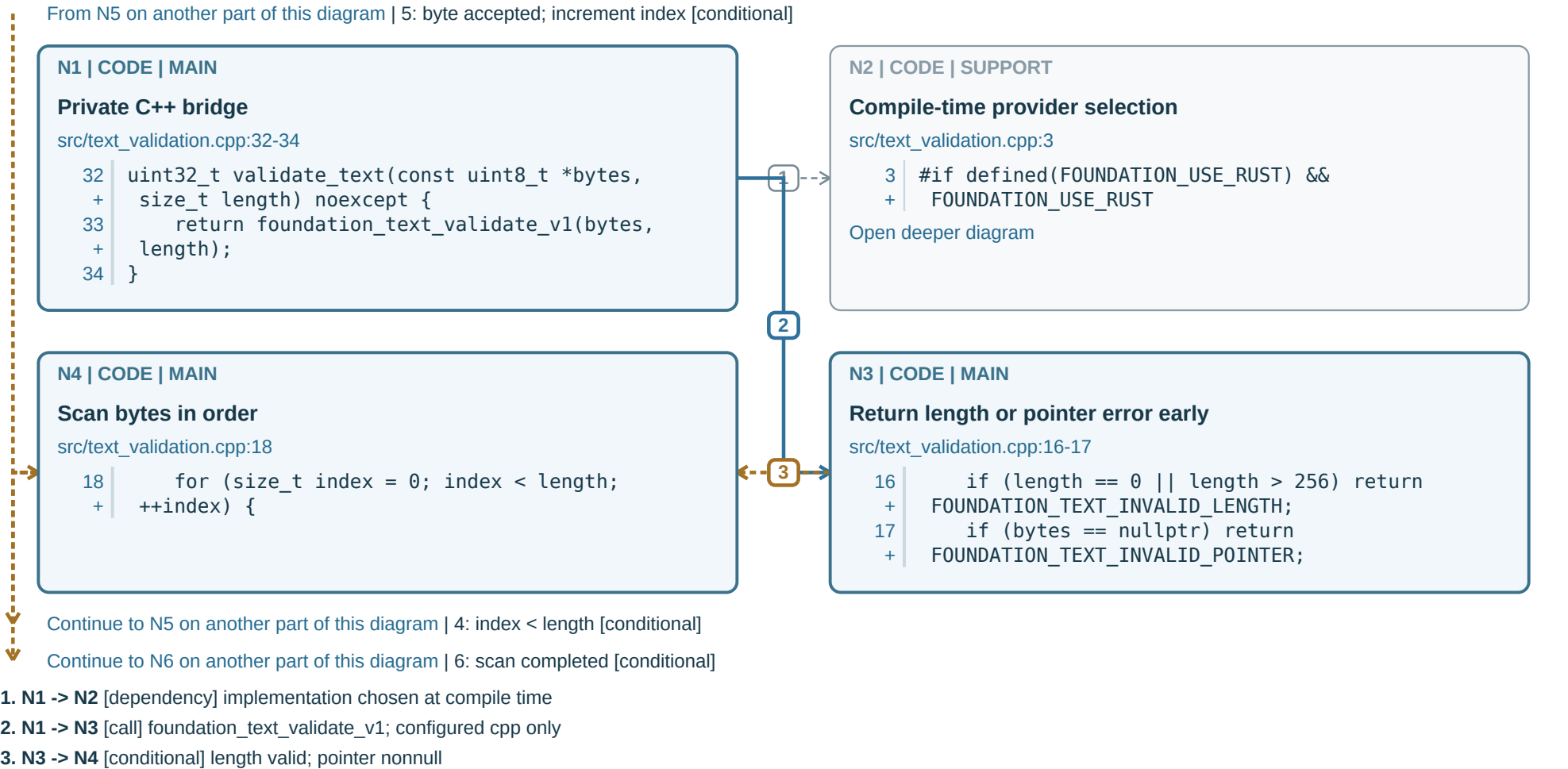


1. N1 -> N2 [step] runtime body reaches bridge call
2. N2 -> N3 [conditional] OK
3. N2 -> N4 [conditional] INVALID_LENGTH
4. N2 -> N5 [conditional] INVALID_ASCII
5. N2 -> N6 [conditional] other status

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [core-validation / status](#) | [Evidence and other parents](#) | [Interactive diagram](#)

Diagram part 1 of 2 | [Next part](#)



[Up: core-validation / status](#) | [Evidence and other parents](#) | [Interactive diagram](#)

[Diagram part 2 of 2](#) | [Previous part](#)

From N4 on another part of this diagram | 4: index < length [conditional]

From N4 on another part of this diagram | 6: scan completed [conditional]

N5 | CODE | MAIN

Reject a non-printable byte

src/text_validation.cpp:19-21

```
19 |         if (bytes[index] < 32 || bytes[index] >
+ |         126) {
20 |             return FOUNDATION_TEXT_INVALID_ASCII;
21 |         }
```

N6 | CODE | MAIN

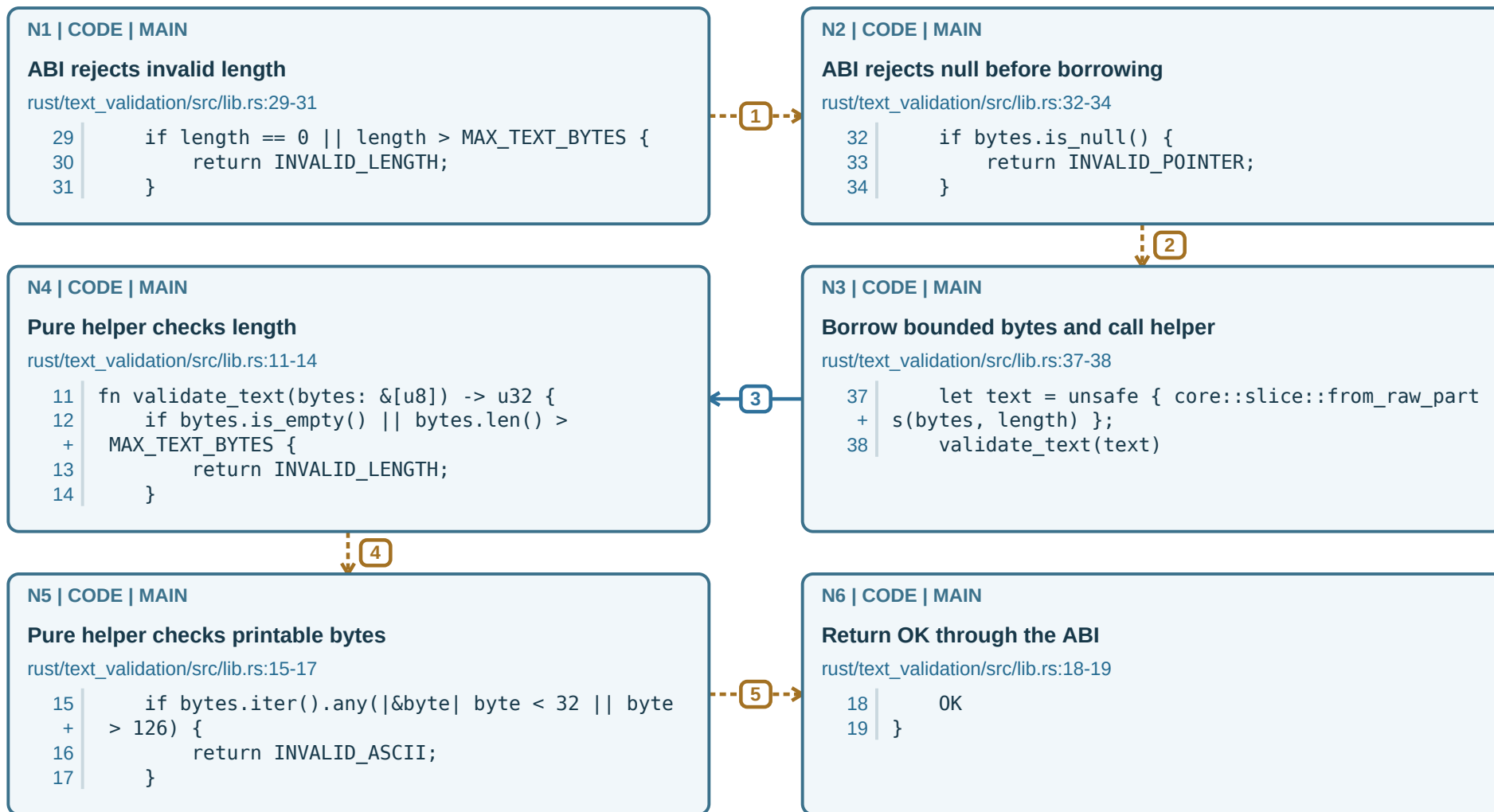
Return OK after the scan

src/text_validation.cpp:23

```
23 |         return FOUNDATION_TEXT_OK;
```

Continue to N4 on another part of this diagram | 5: byte accepted; increment index [conditional]

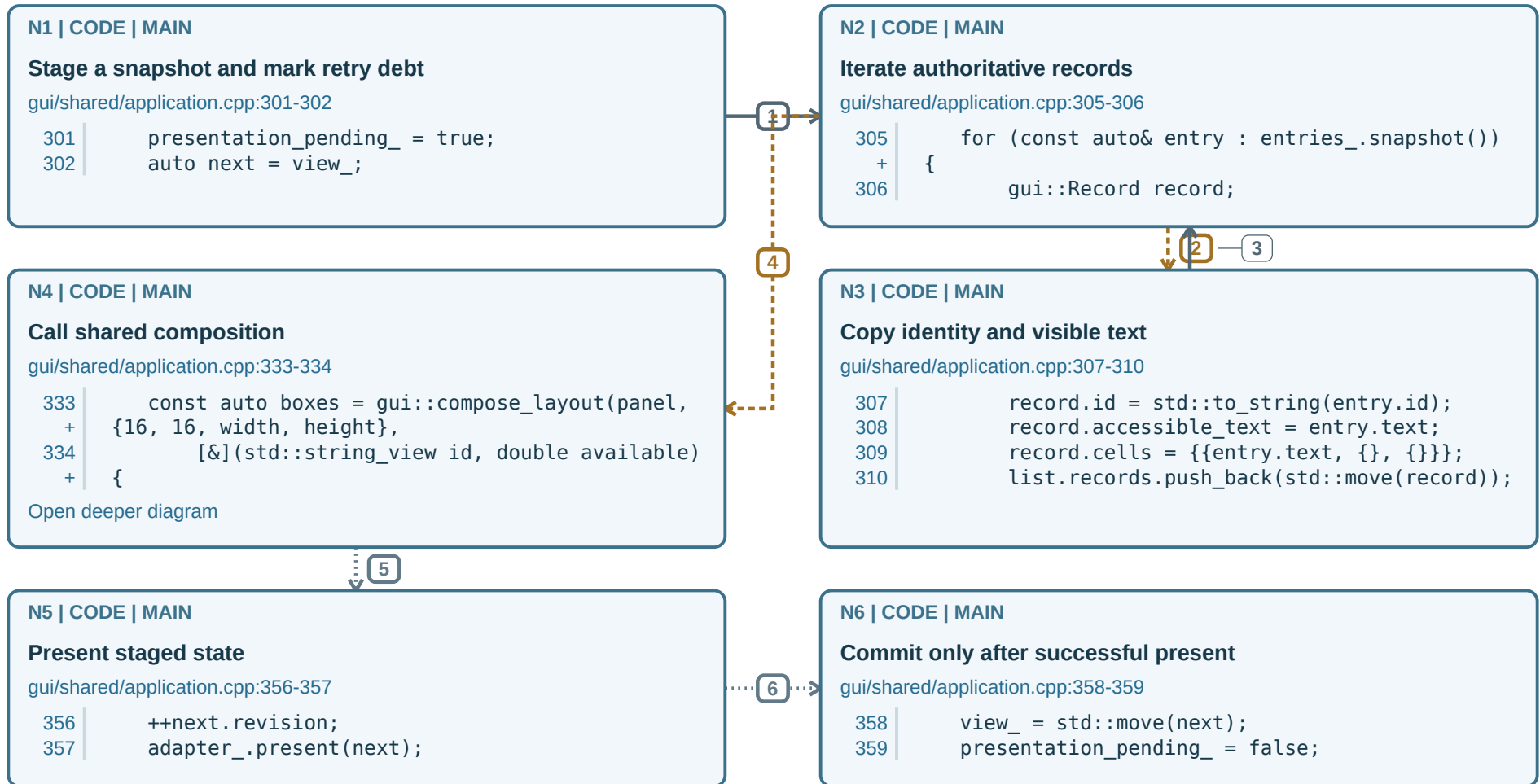
MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [cpp-provider / selection](#) | [Evidence and other parents](#) | [Interactive diagram](#)

1. N1 -> N2 [conditional] length valid
2. N2 -> N3 [conditional] nonnull
3. N3 -> N4 [call] validate_text(text)
4. N4 -> N5 [conditional] length valid
5. N5 -> N6 [conditional] all bytes printable

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: startup | Evidence and other parents | Interactive diagram

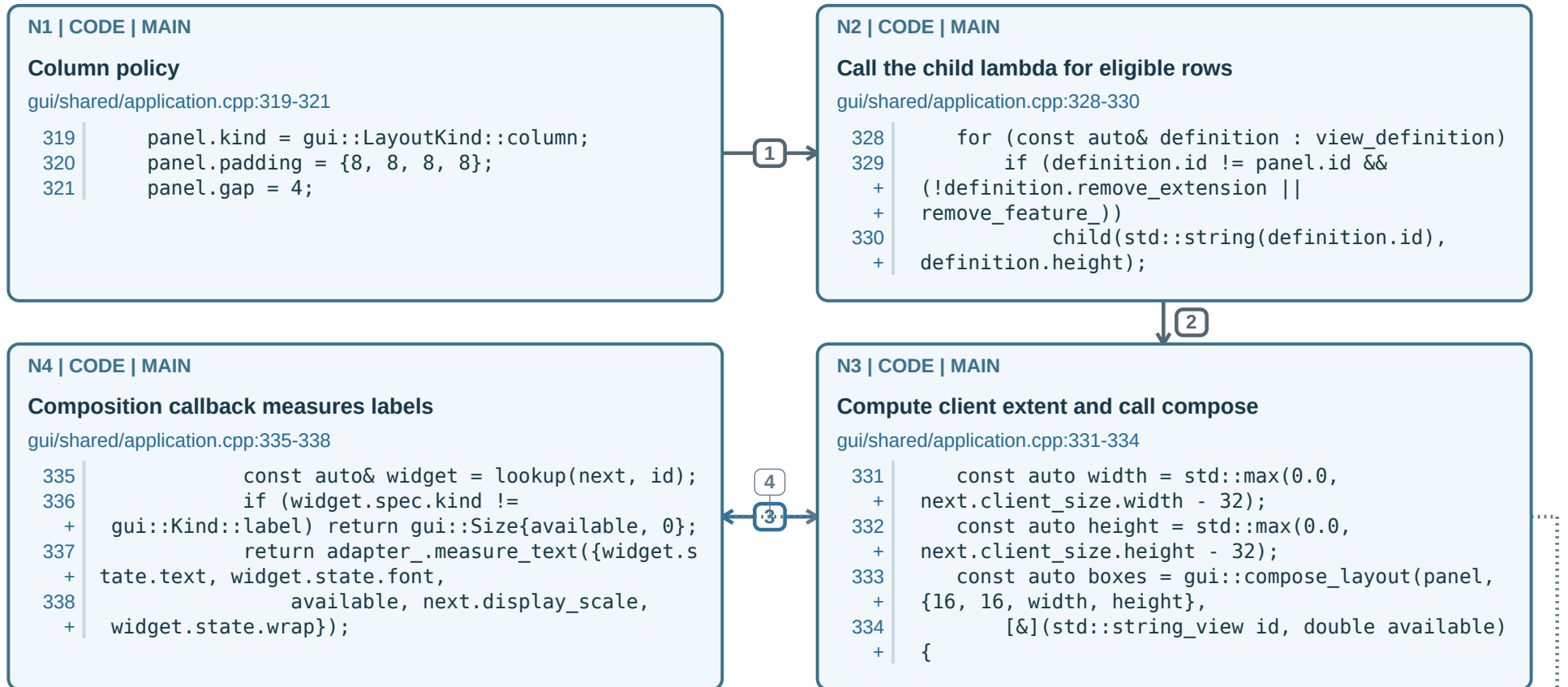


1. N1 -> N2 [step] lookup and clear list; begin iteration
2. N2 -> N3 [conditional] an entry remains
3. N3 -> N2 [step] next entry
4. N2 -> N4 [conditional] no entries remain; status/tree setup follows
5. N4 -> N5 [return] compose returns; bounds and flags updated
6. N5 -> N6 [return] present returned successfully

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [startup](#) | [Evidence and other parents](#) | [Interactive diagram](#)

Diagram part 1 of 2 | [Next part](#)



Continue to N5 on another part of this diagram | 5: compose returns boxes [return]

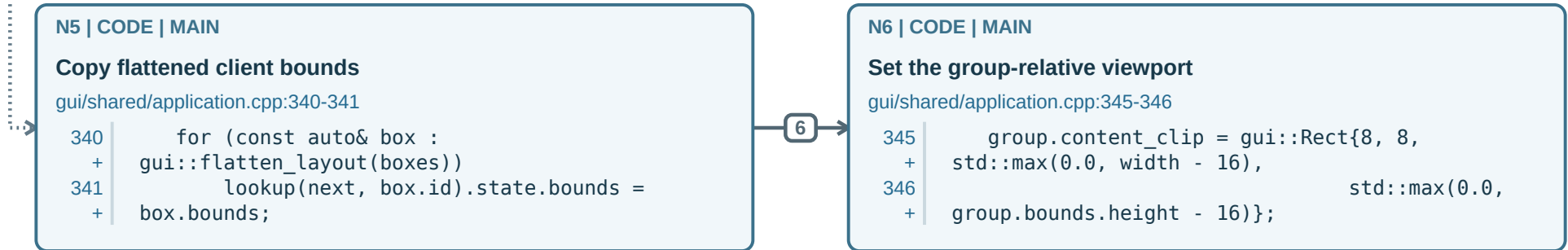
1. N1 -> N2 [step] build the column children
2. N2 -> N3 [step] creation loop completed
3. N3 -> N4 [call] measurement callback as needed
4. N4 -> N3 [return] return measured size

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

[Up: startup](#) | [Evidence and other parents](#) | [Interactive diagram](#)

[Diagram part 2 of 2](#) | [Previous part](#)

[From N3 on another part of this diagram](#) | 5: compose returns boxes [return]



6. N5 -> N6 [step] group extent/height setup follows

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [import](#) | [Evidence and other parents](#) | [Interactive diagram](#)Diagram part 1 of 2 | [Next part](#)

N1 | CODE | MAIN

Select service command and guard identity

gui/shared/application.cpp:133-135

```

133         } else if (input.id == "heading"
+   || input.id == "import" || input.id == "export")
+   {
134             if (next_service_ ==
+   std::numeric_limits<std::uint64_t>::max())
135                 throw
+   std::overflow_error("Service identity
+   exhausted");

```

1

N2 | CODE | MAIN

Create request identity and kind

gui/shared/application.cpp:136-139

```

136         gui::ServiceRequest request;
137         request.id = next_service_;
138         request.kind = input.id ==
+   "heading" ? gui::ServiceKind::prompt :
139             input.id == "import" ?
+   gui::ServiceKind::read_text :
+   gui::ServiceKind::write_text;

```

2

N4 | CODE | MAIN

Export captures record text

gui/shared/application.cpp:143-145

```

143         if (input.id == "export") {
144             for (const auto& record :
+   entries_.snapshot()) request.value += record.text
+   + "\n";
145             if (request.value.size() >
+   request.byte_limit) {

```

3

N3 | CODE | MAIN

Set title and byte bound

gui/shared/application.cpp:140-141

```

140         request.title = input.id ==
+   "heading" ? "Change heading" : input.id ==
+   "import" ? "Import entries" : "Export entries";
141         request.byte_limit = input.id
+   == "heading" ? 80 : 65536;

```

Continue to N6 on another part of this diagram | 4: import / heading; prompt default set if needed [conditional]

Continue to N5 on another part of this diagram | 5: serialized bytes exceed limit [conditional]

Continue to N6 on another part of this diagram | 6: serialized bytes fit [conditional]

1. N1 -> N2 [conditional] service ID available

2. N2 -> N3 [step] next fields

3. N3 -> N4 [conditional] export selected

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [import](#) | [Evidence and other parents](#) | [Interactive diagram](#)

Diagram part 2 of 2 | [Previous part](#)

From N3 on another part of this diagram | 4: [import](#) / heading; prompt default set if needed [conditional]

From N4 on another part of this diagram | 5: [serialized bytes exceed limit](#) [conditional]

From N4 on another part of this diagram | 6: [serialized bytes fit](#) [conditional]

N5 | [CODE](#) | [SUPPORT](#)

Oversize result returns before enqueue

[gui/shared/application.cpp:146-147](#)

```
146 |                                     advance_input_epoch();
147 |                                     status_ = "Export
+ | exceeds the 64 KiB limit"; status_error_ = true;
+ | return;
```

[Open deeper diagram](#)

N6 | [CODE](#) | [MAIN](#)

Advance epoch and enqueue owned request

[gui/shared/application.cpp:150-152](#)

```
150 |                                     advance_input_epoch();
151 |                                     if (!services_.enqueue(std::mo
+ | ve(request)))
152 |                                     throw
+ | std::logic_error("Service request rejected");
```

[Open deeper diagram](#)

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

[Up: import](#) | [Evidence and other parents](#) | [Interactive diagram](#)

[Diagram part 1 of 2](#) | [Next part](#)

[From N6 on another part of this diagram](#) | 6: later tick/input poll or reply [event]

N1 | CODE | SUPPORT

Poll and deliver an existing result

gui/host/contract.hpp:50-51

```
50 |         if (adapter.closed()) { files_.shutdown();  
+   |         return; }  
51 |         if (auto result = files_.poll())  
+   |         current_->complete_service(std::move(*result));
```

[Open deeper diagram](#)

N2 | CODE | SUPPORT

Wait while worker is active

gui/host/contract.hpp:52

```
52 |         if (files_.active()) return;
```

2

N4 | CODE | SUPPORT

Call shared next_service when idle

gui/host/contract.hpp:57

```
57 |         if (!adapter.closed() && !active) if (auto  
+   |         request = application.next_service())
```

N3 | CODE | SUPPORT

Check adapter service/prompt activity

gui/host/contract.hpp:53-56

```
53 |         const bool active = [&] {  
54 |             if constexpr (requires {  
+   |             adapter.service_active(); }) return  
+   |             adapter.service_active();  
55 |             else return bool(adapter.prompt());  
56 |         }();
```

3

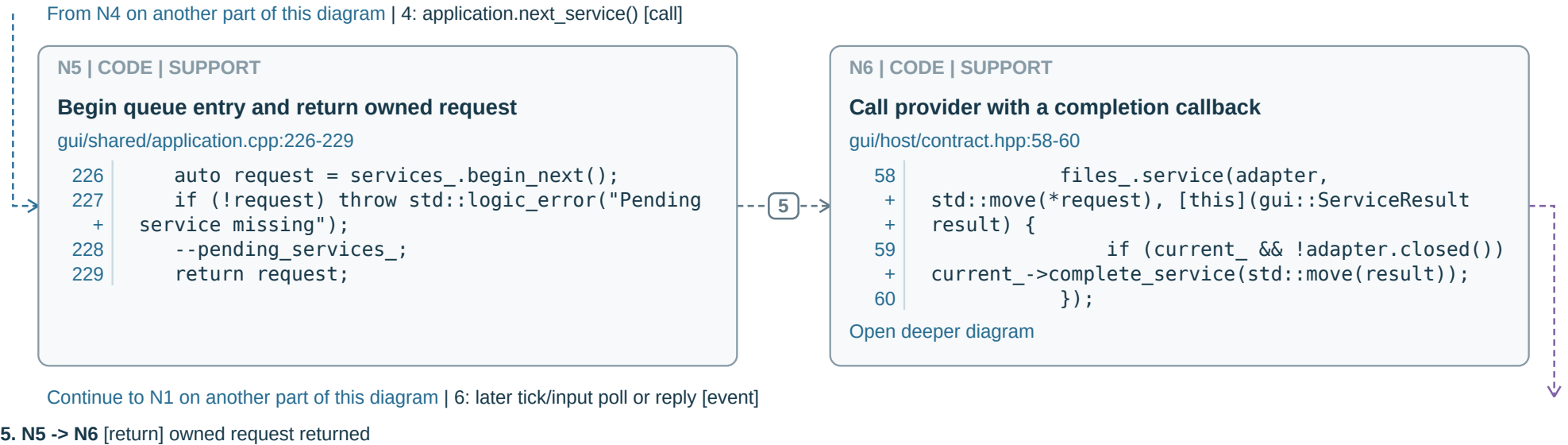
[Continue to N5 on another part of this diagram](#) | 4: application.next_service() [call]

1. N1 -> N2 [step] after optional completion
2. N2 -> N3 [conditional] no active worker
3. N3 -> N4 [conditional] host idle

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

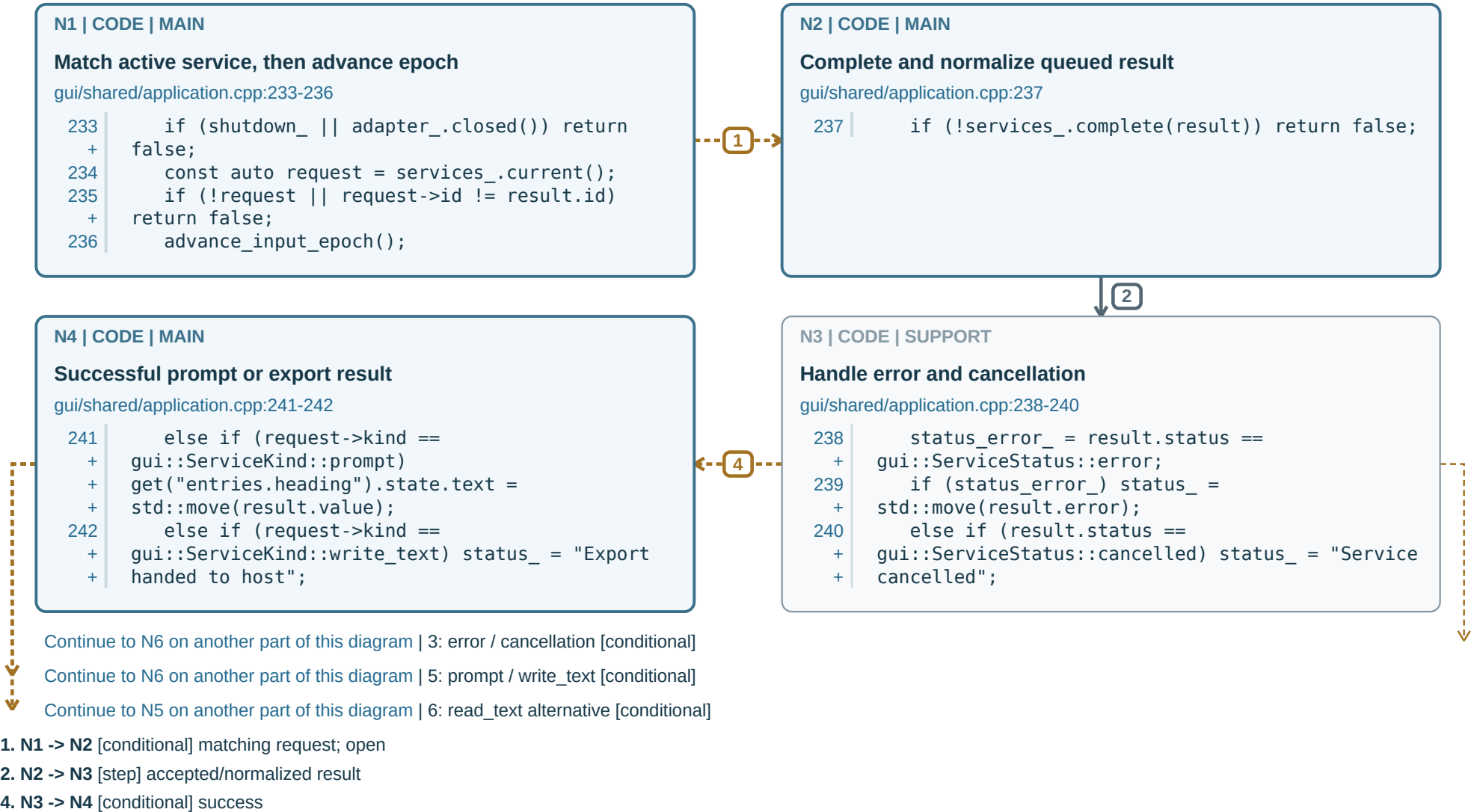
Up: [import](#) | [Evidence and other parents](#) | [Interactive diagram](#)

Diagram part 2 of 2 | [Previous part](#)



MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: import | Evidence and other parents | Interactive diagram
Diagram part 1 of 2 | Next part



MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [import](#) | [Evidence and other parents](#) | [Interactive diagram](#)

Diagram part 2 of 2 | [Previous part](#)

From N3 on another part of this diagram | 3: error / cancellation [conditional]

From N4 on another part of this diagram | 5: prompt / write_text [conditional]

From N4 on another part of this diagram | 6: read_text alternative [conditional]

N5 | CODE | MAIN

Successful read_text enters parser

gui/shared/application.cpp:243

```
243 |         else if (request->kind ==
+ |         gui::ServiceKind::read_text) {
```

[Open deeper diagram](#)



N6 | CODE | MAIN

Catch parser failure, publish accepted outcome

gui/shared/application.cpp:260-263

```
260 |         } catch (const std::exception& error) {
+ |         status_ = error.what(); status_error_ = true; }
261 |         }
262 |         publish();
263 |         return true;
```

[Open deeper diagram](#)

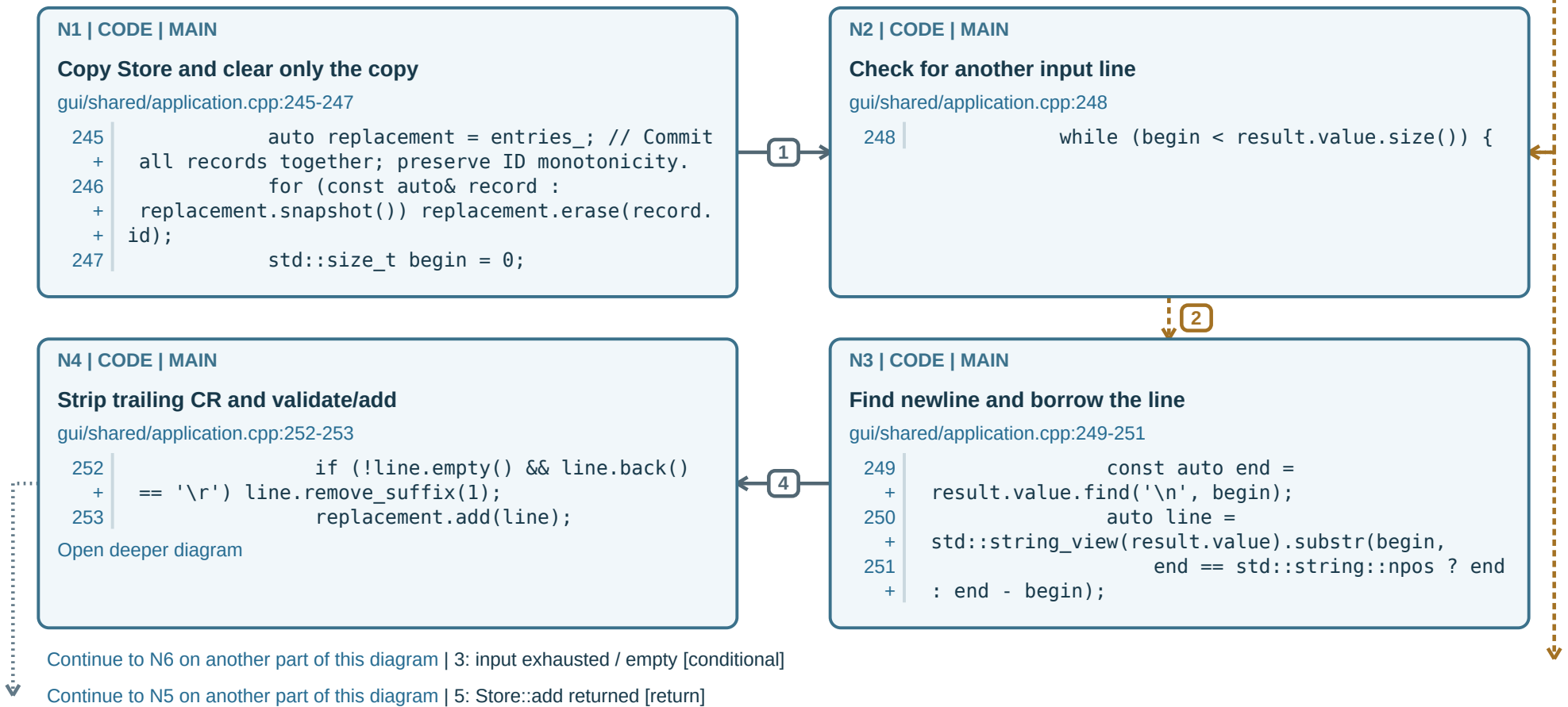
7. N5 -> N6 [return] parser returns or exception is caught

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: import | Evidence and other parents | Interactive diagram

Diagram part 1 of 2 | Next part

From N5 on another part of this diagram | 6: newline found; continue [conditional]

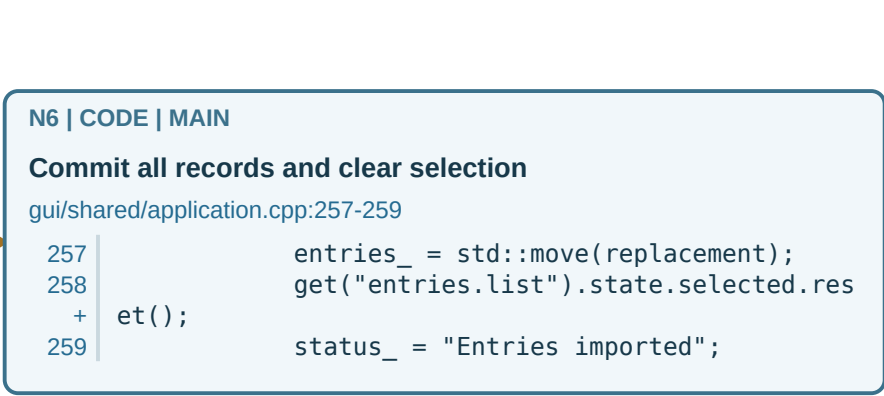
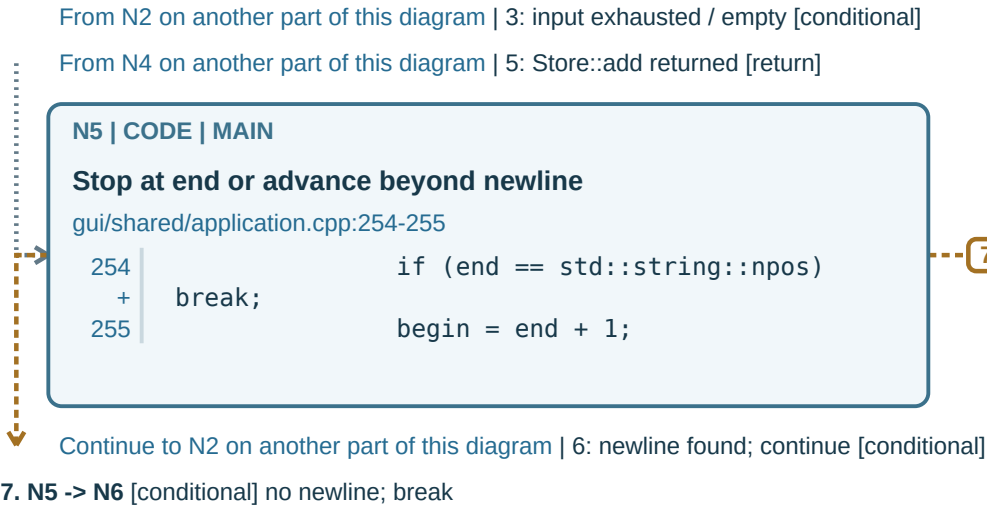


- 1. N1 -> N2 [step] begin staged parsing
- 2. N2 -> N3 [conditional] input remains
- 4. N3 -> N4 [step] line view created

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

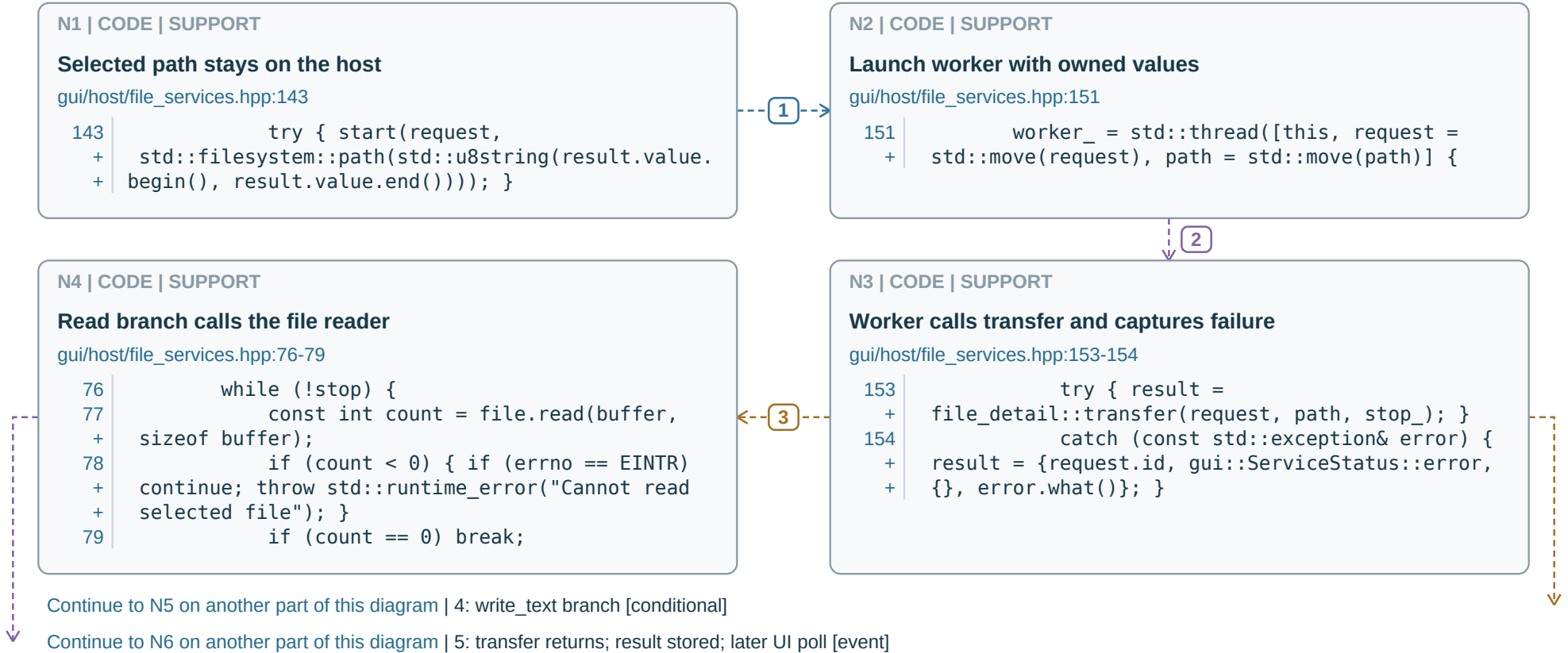
Up: [import](#) | [Evidence and other parents](#) | [Interactive diagram](#)

Diagram part 2 of 2 | [Previous part](#)



MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: [import](#) | [Evidence and other parents](#) | [Interactive diagram](#)
Diagram part 1 of 2 | [Next part](#)



- 1. N1 -> N2 [call] start(request, path)
- 2. N2 -> N3 [event] worker executes its body
- 3. N3 -> N4 [conditional] read_text branch

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Up: import | Evidence and other parents | Interactive diagram

Diagram part 2 of 2 | Previous part

From N3 on another part of this diagram | 4: write_text branch [conditional]
From N4 on another part of this diagram | 5: transfer returns; result stored; later UI poll [event]

N5 | CODE | SUPPORT

Write branch calls the file writer

gui/host/file_services.hpp:99-102

```
99      const int count =
+    file.write(request.value.data() + offset,
100               static_cast<unsigned>(std::min<std
+    ::size_t>(4096, request.value.size() - offset)));
101      if (count < 0 && errno == EINTR)
+    continue;
102      if (count <= 0) throw
+    std::runtime_error("Cannot write selected file");
```

N6 | CODE | SUPPORT

UI poll retrieves result and joins worker

gui/host/file_services.hpp:160-162

```
160      { std::lock_guard lock(mutex_);
+    result.swap(result_); }
161      if (result && worker_.joinable())
+    worker_.join();
162      return result;
```

Open deeper diagram

6. N5 -> N6 [event] transfer returns; result stored; later UI poll

MAIN = emphasized path; SUPPORT / dashed dependency = supporting work. '+' = wrapped source continuation. Literal excerpts; surrounding code remains linked.

Diagram evidence and scope

These notes preserve the selected-path boundaries and source context. Use the return link to resume the graphical diagram; complete source bodies remain in the implementation handbook and explorer.

main to the declared widgets: calls and the data they consume

[Return to code diagram](#) | [Up: startup](#)

Normal native startup reaches the shared Application constructor, which loops over view_definition and calls add for each eligible row to create shared Snapshot widgets. The editor/Add declarations feed that loop as data; they are not calls. Layout and adapter presentation happen later in publish.

Scope: Primary native startup to shared Snapshot widget creation

N2: Session constructor -> Application	The runner creates NativeSession, a Session alias. The callback body runs later; application(adapter) constructs the shared Application now. Implementation PDF p22
N3: Enter Application constructor	Implementation PDF p20
N4: view_definition: editor and Add rows	These constexpr aggregate values are declarations. The same table supplies creation order and later layout policy.
N5: Creation loop -> add(definition)	Only rows with remove_extension == false are created here. Its declared default is false; the optional Remove row sets it true. Implementation PDF p20
N6: Application::add creates one widget	The remainder copies strings/font/wrapping, appends to the shared view_ Snapshot, and returns the widget. Height is consumed later by publish/layout; add creates no native control. Implementation PDF p31

Widget declaration table: fields, rows and their consumer

[Return to code diagram](#) | [Up: startup](#) | [Up: main-to-widgets / definitions](#) | [Up: main-to-widgets / loop](#) | [Up: application-init / rows](#)

Representative literal rows from the application-owned table. Aggregate field order and defaults expand from the first card. Data arrows feed the actual constructor loop; only its add arrow is a call. The constructor excludes the optional Remove row.

Scope: Declaration data and constructor consumer

N1: ViewDefinition aggregate field order	Following fields declare bold=false, wrap=none and remove_extension=false. Expand for their exact declarations. Implementation PDF p4
N2: First row declares the parent group	add compares each ID with view_definition.front().id; the first root gets no parent assignment. Implementation PDF p119
N3: Editor and Add declaration values	Other ordinary rows are omitted here. Their table order is preserved by the creation loop.
N4: Optional Remove row sets the final flag	The final true initializes remove_extension. This row is skipped during default constructor creation.
N5: Constructor iterates the declaration table	Implementation PDF p20

N6: add copies identity and chooses the default parent	Non-root IDs receive the first row as parent. Height is used later by publish/layout; the loop already filtered remove_extension. Implementation PDF p31
--	---

Shell entry, Python dispatch and process plumbing

[Return to code diagram](#) | [Up: compile](#) | [Up: test](#)

The primary route is shell exec -> argument parser -> execute. The supporting cards show the timing/process helpers used by execute; the connecting label explicitly skips into execute rather than implying main calls them directly.

Scope: Literal wrapper entry and representative non-Windows subprocess boundary.

N2: main declares action and preset arguments	The remaining parser options continue in the source; this is the shared build/test/package entry.
N4: PhaseTimings.call calls its supplied function	The opt-in timing branch below calls the same function and records its duration.
N6: Non-Windows branch launches the child process	The Windows branch continues below with its owned BuildSession/process_tree launcher.

execute: configure, select, compile and dispatch

[Return to code diagram](#) | [Up: compile](#) | [Up: test](#) | [Up: build-entry / execute](#) | [Up: release-package / test-command](#)

These are ordered statements and conditional dispatch inside execute. Input selection, identity guards, cache stamps and some command arguments are omitted between cards. CMake configuration does not itself launch the next build command.

Scope: Wrapper statement order; not a configured target execution trace.

N1: Construct the configure argument list	The literal list continues with other flags, then selected SDK/toolchain arguments. Values remain symbolic.
N2: Launch CMake configuration	Earlier source/input checks precede this call; cache identity is written immediately afterwards.
N3: Test branch chooses exact or aggregate targets	This block is inside action==test; the preceding default is targets=["all"].
N4: Launch the selected prerequisite build	Generated build tools schedule target prerequisites. The wrapper checks source/selection stability next.
N5: Test branch launches CTest	Only action==test reaches this call; command is constructed above with selections and optional JUnit. Package is a separate elif branch.
N6: Final source check after test/package writers	Plain build already checks source immediately after compilation. SDK/Rust/dependency checks continue below before return 0.

The wrapper rechecks inputs before its success/error exit

[Return to code diagram](#) | [Up: compile](#) | [Up: test](#) | [Up: build-sequence / final-source](#)

The post-compile source check applies to every action. Test/package run additional writers and therefore have a second source observation. Selected dependency/tool checks follow; failures throw to the CLI error boundary.

Scope: Real terminal source checks and representative input/failure boundaries; omitted guards remain in source.

N1: Every action rechecks source after compilation	The named-test branch also compares configured selection/prerequisites immediately before this check.
--	---

N2: Test/package recheck after their later execution	This guard deliberately excludes plain build, whose terminal source check is the preceding card.
N3: Recheck the selected prepared SDK	GUI input-group verification precedes this card when selected.
N4: Native Rust branch rechecks selected tool identity	This elif follows the prepared Rust SDK check. The remaining Windows/native-prefix/Wasm checks continue below.
N5: execute returns zero after all selected checks	The module entry calls sys.exit(main()); main returns execute's result and finishes optional timing output.

CMake declarations: provider, core sources and CLI link

[Return to code diagram](#) | [Up: compile](#) | [Up: build-sequence / configure-argv](#) | [Up: build-sequence / configure](#) | [Up: build-sequence / compile](#)

Cards show real configuration statements and declared relationships. Dependency arrows mean source/link/provider relationships, not compilation order. Variables, platform conditions, options helpers and generator expressions are not evaluated.

Scope: Configuration declarations and target relationships; no compile/link chronological ordering.

N1: Project language and configured test support	The file includes CTest and build helpers later; the selected preset/toolchain establishes actual compiler configuration.
N2: Provider cache selection	Fresh configuration defaults to Rust. Validation and frozen-provider checks follow these declarations. Implementation PDF p20
N3: Declare the C++ core and alias	Store and its C++ boundary remain compiled with either text-validation provider. Implementation PDF p4
N4: Publish core header search paths	Generator expressions distinguish build and installed consumers; they stay literal in this chart.
N5: Declare CLI sources and its core link	foundation_options adds selected platform/runtime/warning flags; optional GUI targets have separate declarations. Implementation PDF p4

CMake selects and declares the Rust component

[Return to code diagram](#) | [Up: compile](#) | [Up: cmake-core / provider](#) | [Up: cmake-core / rust-helper](#)

The non-Rust branch returns. For Rust, omitted platform/SDK checks resolve symbolic tools and configuration; describe runs during configuration, while build/verify commands are deferred to the generated custom target.

Scope: Configure-time commands versus deferred build-target commands.

N2: Declare an imported static archive target	Platform/SDK/tool identity checks precede this declaration; each configuration gets an imported location. Implementation PDF p6
N3: Configuration invokes Rust input description	This describe subprocess is configuration work, not Cargo compilation.
N4: Declare deferred build and verify commands	BYPRODUCTS and VERBATIM complete this declaration below. Commands run when the configured target is built.
N5: Imported archive requires the custom target	This is a generated target dependency, not a configure-time call to Cargo.
N6: Select the private ABI provider and core link	The C++ core remains the owner of Store; only its private text validator is supplied by Rust.

Rust custom target enters the frozen Cargo builder

[Return to code diagram](#) | [Up: rust-component / custom-target](#)

The custom target process dispatches build(config), which resolves captured configuration and invokes Cargo. Prior-receipt/cache checks and artifact validation are omitted between short cards; verify is a separate subsequent custom command.

Scope: Actual builder code and the separate generated verification command.

N2: Load checked build configuration	Previous receipt and input identity checks continue before Cargo is invoked.
N3: Create a private selected-tool environment	Changed identified inputs also invalidate the private Cargo package cache.
N4: Construct the literal Cargo build command	The release profile condition appends --release below; all tool/target paths remain symbolic here.
N5: Invoke Cargo through _run	The builder then validates parsed compiler-artifact output and writes its bound receipt. Cargo test is a different command.
N6: CMake invokes the separate verifier afterwards	This card returns to the generated custom target declaration; it is not a direct Python call from build to verify.

Configured test commands and their compile prerequisites

[Return to code diagram](#) | [Up: test](#) | [Up: build-sequence / selection](#) | [Up: build-sequence / ctest](#)

CTest command declarations and foundation_test_prerequisites registration are configuration relationships. The wrapper builds selected registered targets before CTest launches a child. These examples do not enumerate every test.

Scope: Configured declarations/prerequisites; actual child entry code is one level below.

N1: Declare the native core test executable	The native branch uses this executable. Browser configuration uses its retained Node executor. Implementation PDF p5
N2: Register core.store command	This declaration tells CTest what to launch when native core.store is selected.
N3: Register core.store compile prerequisites	Registration does not execute the test or compile it.
N4: Declare aggregate prerequisite targets	Finalization validates registrations and writes test-prerequisites.json. Actual lists/labels are configured values.
N5: Register a tools.* Python suite command	The CMake loop supplies the registered suite; this chart does not expand that inventory.
N6: Register exact-host native Rust unit tests	Conditional rust.unit calls the distinct Cargo test path; it is not the custom target Cargo build command.

core.store enters the C++ contract test

[Return to code diagram](#) | [Up: test](#) | [Up: test-registration / core-command](#)

A representative path through the existing native test main. The full test includes more invalid-input and all-byte checks between excerpts. Assertions/helpers can throw; the enclosing catch maps failures to exit 1.

Scope: Representative existing C++ test statements, with omitted assertions named explicitly.

N1: Native executable enters main	rejects executes its callable and expects the specified exception.
-----------------------------------	--

N3: Check rejected replacement input	Additional ASCII/NUL/missing-ID checks continue below; these are literal test calls, not proposed examples.
N6: Success output or exception exit	The all-byte loop precedes this excerpt. Falling out of main after success returns zero.

tools.* enters the recorded unittest suite runner

[Return to code diagram](#) | [Up: test](#) | [Up: test-registration / python-command](#)

The selected suite name resolves an existing module, which is loaded before execute runs the discovered unittest suite. Recorded outcomes must cover every applicable required test; platform exclusions are explicit.

Scope: Existing Python suite entry, dynamic module selection and actual unittest call.

N1: Resolve the suite after CLI parsing	suite_source validates the registered name and native diagnostic platform, then resolves an existing module path.
N2: Load that existing test module	Only the application runner executes test modules. The documentation generator reads their captured source.
N4: execute validates the test inventory	The omitted body rejects empty/duplicate inventories and separates explicitly inapplicable tests.

Manual and hosted release entry boundaries

[Return to code diagram](#) | [Up: release](#)

Manual RELEASE is a procedure, not an executable Python entry. The hosted ordinary route calls reusable workflows, requires full regression before assembly/publication, and schedules certification/promotion only for execute-enabled requests. Workflow dependency arrows are scheduling constraints, not function calls.

Scope: Manual alternative plus hosted scheduling declarations; local work still runs in preparation-only mode.

N1: Manual procedure starts with frozen inputs	This is the alternative local procedure; its later publication step is separately authorized.
N3: Ordinary candidate requires full regression	The reusable application workflow launches nested candidate regression. execute gates remote writes, not local production.
N5: Hosted certification follows the candidate and receipt gate	The top-level regression job checks the nested regression receipt; it does not rerun the suite.

Prepared producer: full test, package, archive verification

[Return to code diagram](#) | [Up: release](#) | [Up: release-entry / manual](#) | [Up: release-entry / hosted](#)

The normal prepared producer runs a full release test action, then a separate package action. Windows graphics production has an explicit alternate qualification wrapper. CPack and archive verification are separate calls.

Scope: Actual prepared producer and its package subprocess; not candidate publication.

N1: Prepared producer constructs the full test command	Selected verified SDK/provider/GUI flags are appended below before execution.
N2: Normal branch executes that test action	The preceding graphics-needed branch runs windows_gui_qualification instead.
N3: Change a copied command to package	The next statements also remove the test-only --junit argument/value.
N4: Execute package action and select the platform archive	The producer requires exactly one archive and copies it into the output handoff.

N5: Inside build.execute: launch CPack	The package wrapper first configures/builds. --verify-package is its separate opt-in subprocess verification branch.
N6: Back in producer: describe and verify copied archive	These helper calls do not publish or certify. Further source/provider identity checks and artifact.json follow.

Assemble complete local releases from existing archives

[Return to code diagram](#) | [Up: release](#) | [Up: release-entry / assembly-gate](#)

ci_plan prepares an exact assembly specification and calls release.assemble. The assembler stages existing source, application archives and dependency groups, verifies the inventory, and renames new staging into place. No compile/publish call exists here.

Scope: Actual local helper calls/statements with staging and loop steps explicitly omitted.

N1: Prepared assembly helper calls release.assemble	The omitted earlier helper code verifies complete target/backend inventory before writing spec.
N2: Assembler rejects reused output paths	assemble reads the spec first and creates temporary sibling staging after this guard.
N3: Nested retain copies and checks exact input bytes	retain is called for source and archive/manifest files. Source hash/archive verification follows before artifact processing.
N4: Validate each retained application package	The surrounding artifact loop first compares the archive digest with its frozen entry.
N6: Inventory, verify and commit the local directory	The output remains a local candidate. A separate caller may subsequently plan or execute remote publication.

Candidate publication has a plan branch and an execute branch

[Return to code diagram](#) | [Up: release](#)

The lifecycle dispatcher explicitly calls execute=True for publication. Other callers can receive a plan. The executing helper creates a fresh draft/tag, uploads and verifies the draft bytes, then publishes a prerelease with Latest false.

Scope: Actual remote mutation branch; prerelease publication does not promote Latest.

N1: Explicit hosted publication invokes the helper	The surrounding publish-candidate dispatcher reads the frozen request and checks source/packager revision first.
N2: The helper returns early unless execute is true	The function default is execute=False. Local assemble calls it without execute to record a plan.
N3: Executing branch creates a fresh tag and draft	Existing releases/tags are rejected before these writes. The remote draft is observed before upload.
N4: Upload immutable candidate payloads	The bound delivery.json descriptor is uploaded next; payload upload batches are bounded.
N5: Verify the uploaded draft identity and bytes	Here readback keeps its default true: verified_remote downloads payloads and verifies the reconstructed release.
N6: Publish prerelease and ensure it is outside Latest	Certification is a subsequent workflow/manual operation, not a call from publish_candidate.

Qualify exact bytes, calculate eligibility and attach an attempt

[Return to code diagram](#) | [Up: release](#) | [Up: release-entry / certification](#) | [Up: release-publish / prerelease](#)

Physical qualification and certificate calculation are separate dispatched workflow operations with reports handed across jobs. The certificate binds source/inventory/policy/environment coverage. Attachment appends an immutable attempt; it does not change Latest.

Scope: Grouped physical execution, exact certificate calculation and separate immutable attachment boundary.

N1: Dispatcher invokes prepare or run for a physical check	The plan/matrix assigns each physical execution its required environment. This is not one local test loop.
N2: Certificate dispatcher calls certify	The dispatcher gathers exact logical report paths first; certify checks their bound subject and required scopes.
N3: certify merges reports and checks actual host evidence	Source/inventory/policy and target/backend/scope guards precede this excerpt.
N4: Returned certificate marks promotion eligibility	passed is derived from merged coverage. The return also records exact subject, reports, run identity and limitations.
N5: A separate dispatcher operation executes attachment	The certificate operation previously planned attachment; this branch explicitly executes the append-only write.
N6: Attachment helper defaults to planning	It validates/bundles the exact attempt before remote attachment. Promotion is a separate caller after successful handoff.

Exact certificate verification precedes the Latest mutation

[Return to code diagram](#) | [Up: release](#) | [Up: release-entry / promotion](#) | [Up: release-certify / attachment-helper](#)

A separate promotion dispatcher chooses execute. The helper rejects unsupported subjects, revalidates remote assets and the exact certificate, changes Latest, then reads it back. Hosted metadata-only handoffs use verified API digests and bound evidence.

Scope: Actual promotion branch plus separately scheduled final verification.

N1: Promotion dispatcher explicitly selects plan or execution	request above binds run/attempt/certificate SHA and sets metadata_only=True. promotion-plan leaves execute false.
N2: Promote rejects experiment/base/mismatched subjects	Exact run/attempt/certificate-SHA validation follows. execute=False returns a plan before remote mutation.
N3: Executing branch revalidates assets and certificate	Remote identity and policy are checked again before changing Latest; metadata_only controls repeated payload downloads.
N5: Read back and verify the Latest pointer	Promoted lifecycle, assets and tag are also rechecked immediately before this excerpt.
N6: Hosted final job independently verifies the outcome	This is a later workflow process, not a call from promote. Preparation-only runs verify that remote publication was not requested.

ViewDefinition: literal types and declared defaults

[Return to code diagram](#) | [Up: widget-definitions / fields](#)

Field declarations in aggregate order. Dependency arrows show declaration order, not execution. The editor/Add rows supply the first six fields and use the trailing declared defaults. Other rows can override those defaults, as the optional Remove row does.

Scope: Application-owned declaration types and defaults

N1: Identity, widget kind and layout height	Implementation PDF p4
N2: Text, label and placeholder	Implementation PDF p4
N3: Default font weight and wrapping	Implementation PDF p4

N4: Default creation flag	The constructor creates false rows. The optional Remove declaration explicitly overrides this default with true. Implementation PDF p4
---------------------------	---

Native GUI startup: real entry and host calls

[Return to code diagram](#) | [Up: startup](#) | [Up: main-to-widgets / entry](#)

Normal FLTK launch, excluding the qualification branch. Expand construction for shared behavior. Window waiting and timer callbacks are supporting host infrastructure.

Scope: Native composition; supporting host infrastructure

N2: Runner constructs its session	Argument checks run before this statement; the smoke branch is outside this normal-launch diagram.
N3: Session constructs callback and Application	The callback body runs on later input. application(adapter) constructs the shared App; current_ is assigned afterward. Implementation PDF p22
N6: Timer calls the session	Session::tick first advances Application work, then optional adapter sync, then services. Closing/error cleanup is outside this excerpt.

Object relationships: Session, Application and Store

[Return to code diagram](#) | [Up: main-to-widgets / session](#)

These are declaration relationships, not runtime call arrows. Session contains its adapter and Application. Application borrows the adapter by reference, owns a Snapshot and owns its Store; the constructor body expands separately.

Scope: Object ownership and declaration relationships

N3: Session supplies adapter to Application	Implementation PDF p22
N4: Application borrows adapter and owns view	Implementation PDF p14
N5: Application owns a bounded Store	The Store member is constructed before the Application constructor body; 1000 is the current application capacity. Implementation PDF p14
N6: Store owns records and next identity	Implementation PDF p18

Application constructor: declarations to first publish

[Return to code diagram](#) | [Up: startup](#) | [Up: main-to-widgets / application](#) | [Up: gui-startup / construct](#) | [Up: application-ownership / construct](#)

Existing constructor statements. The default executor argument is evaluated before entry; this diagram begins with the received executor. Widget creation and publication expand below.

Scope: Shared application behavior

N1: Bind supplied state	Implementation PDF p20
N2: Require executor and set title	Implementation PDF p20
N3: Create eligible declaration rows	Implementation PDF p20

N4: Configure editor input	Implementation PDF p20
N5: Declare menu options	Implementation PDF p20
N6: Configure list and publish	Implementation PDF p20

Application::add: create one declared widget

Return to code diagram | Up: main-to-widgets / add | Up: widget-definitions / add

Called once per eligible row. The parent condition applies to every row except the current first root. Return resumes the constructor or optional-feature creation loop.

Scope: Shared application behavior

N1: Create widget and identity	Implementation PDF p31
N2: Assign the current root parent	Implementation PDF p31
N3: Copy presentation strings	Implementation PDF p31
N4: Copy font and wrapping	Implementation PDF p31
N5: Append owned widget	Implementation PDF p31
N6: Return the created widget	Implementation PDF p31

Add-button dispatch: callback to helper and publication

Return to code diagram | Up: button-click | Up: service-request / oversize

Selected WidgetEvent/Activate path. Close, resize, other inputs and other target IDs have their own branches in the full handler. Rejected normalization returns before publication.

Scope: Shared application behavior

N1: Native callback calls Application	services() runs after handle returns. Browser forwarding uses Browser<App> instead of this Session callback. Implementation PDF p22
N2: Reject closed adapter	Implementation PDF p5
N3: Normalize current input	The close-event and shutdown branches precede this fragment. Scroll lookup is a callback used by normalization. Implementation PDF p5
N4: Enter the widget visitor	Implementation PDF p5
N5: Route the Add target	Implementation PDF p5
N6: Publish after normal visitor return	Implementation PDF p5

append_entry: epoch, core call, success or caught error

Return to code diagram | Up: button-click | Up: event-dispatch / activation

The semantic epoch advances before mutation and outside the try block. Caught invalid_argument and length_error return to handle; other exceptions can escape and skip its publication.

Scope: Shared application behavior

N1: Advance epoch and read editor	Implementation PDF p11
N2: Call the domain operation	Implementation PDF p11
N3: Successful input clears editor	Implementation PDF p11
N4: Caught input rejection	Implementation PDF p11
N5: Caught capacity failure	Implementation PDF p11
N6: Epoch helper rejects exhaustion first	Called by the first block; the overflow exception is outside append_entry's catches. Implementation PDF p87

Store::add: validate, capacity, identity, insertion

[Return to code diagram](#) | [Up: button-click](#) | [Up: append-entry / core](#) | [Up: import-parser / add](#)

Failed validation, capacity, identity or allocation exits before ID advancement. Throw statements inside guard boxes are terminal exits; arrows show the accepted path.

Scope: Reusable domain behavior

N1: Validate before mutation	Implementation PDF p23
N2: Check collection capacity	Implementation PDF p23
N3: Check identifier availability	Implementation PDF p23
N4: Insert an owned record	Implementation PDF p23
N5: Advance ID after insertion	Implementation PDF p23
N6: Return stable identity	Implementation PDF p23

Store::validate: ABI status to C++ contract

[Return to code diagram](#) | [Up: button-click](#) | [Up: core-add / validate](#)

The assertion is compile-time context. Runtime calls the selected provider bridge, then chooses one switch case. Invalid-pointer or unknown status reaches default, rather than ordinary input rejection.

Scope: Reusable domain validation

N1: Declaration and compile-time bound	static_assert is not an executing runtime branch. Implementation PDF p72
N2: Call private validation and switch on status	Implementation PDF p72
N3: Valid input returns	Implementation PDF p72
N4: Translate length rejection	Implementation PDF p72

N5: Translate byte rejection	Implementation PDF p72
N6: Reject component failure	Implementation PDF p72

Private ABI bridge and the selected C++ provider

[Return to code diagram](#) | [Up: core-validation / status](#)

The bridge calls one configured implementation. The preprocessor box is a declaration dependency, not a runtime decision; expand it for the mutually exclusive Rust implementation. Guard returns leave the provider.

Scope: Private provider boundary

N1: Private C++ bridge	Implementation PDF p116
N2: Compile-time provider selection	Rust is selected when this condition is true; the C++ ABI implementation below is in its #else branch.
N3: Return length or pointer error early	Implementation PDF p76
N4: Scan bytes in order	Implementation PDF p76
N5: Reject a non-printable byte	Implementation PDF p76
N6: Return OK after the scan	Implementation PDF p76

Selected Rust provider: preflight, borrow, pure checks

[Return to code diagram](#) | [Up: cpp-provider / selection](#)

This alternative is called by the same private bridge when configured for Rust. Length and null checks precede slice creation. Each guard can return its status; the helper length check is redundant after ABI preflight.

Scope: Private Rust provider alternative

N1: ABI rejects invalid length	Implementation PDF p117
N2: ABI rejects null before borrowing	Implementation PDF p117
N3: Borrow bounded bytes and call helper	The caller must supply initialized bytes in one unchanged live allocation; this function retains none. Implementation PDF p117
N4: Pure helper checks length	Implementation PDF p74
N5: Pure helper checks printable bytes	Implementation PDF p74
N6: Return OK through the ABI	Implementation PDF p74

Publication: authoritative records to adapter, then commit

[Return to code diagram](#) | [Up: startup](#) | [Up: button-click](#) | [Up: application-init / publish](#) | [Up: event-dispatch / publish](#) | [Up: service-completion / publish](#)

Selected critical statements from publish. List lookup/clear, status, enabled flags and scroll geometry also run between the highlighted groups. A presentation exception skips the final view commit and leaves retry debt.

Scope: Shared application behavior

N1: Stage a snapshot and mark retry debt	Implementation PDF p26
N2: Iterate authoritative records	Implementation PDF p26
N3: Copy identity and visible text	Implementation PDF p26
N4: Call shared composition	Implementation PDF p26
N5: Present staged state	Implementation PDF p26
N6: Commit only after successful present	Implementation PDF p26

Shared layout: rows, composition callback, bounds and clip

[Return to code diagram](#) | [Up: startup](#) | [Up: publish-view / layout](#)

The panel is created with ID entries.form before these policy statements. The child lambda builds one LayoutNode per eligible row. Supplier compose internals are a boundary; the captured callback and output copying are shown.

Scope: Shared application behavior

N1: Column policy	Implementation PDF p26
N2: Call the child lambda for eligible rows	Implementation PDF p26
N3: Compute client extent and call compose	Implementation PDF p26
N4: Composition callback measures labels	Implementation PDF p26
N5: Copy flattened client bounds	Implementation PDF p26
N6: Set the group-relative viewport	Group content size and bounded height are assigned immediately before this fragment. content_clip coordinates are group-relative; widget bounds are client-relative. Implementation PDF p26

Menu service request: fields, export bound, enqueue

[Return to code diagram](#) | [Up: import](#) | [Up: export](#)

Shared heading/import/export branch after normalized ChooseOption. Import skips export serialization. A return in the oversize box returns from the visitor; handle still publishes. Enqueue success updates counters and normal handle publication precedes host service pumping.

Scope: Shared application behavior

N1: Select service command and guard identity	Implementation PDF p5
N2: Create request identity and kind	Implementation PDF p5

N3: Set title and byte bound	read_text/write_text carry valid UTF-8 text with a 64 KiB contract maximum, not arbitrary binary content. Implementation PDF p5
N4: Export captures record text	Implementation PDF p5
N5: Oversize result returns before enqueue	Implementation PDF p5
N6: Advance epoch and enqueue owned request	Pending/service-ID counters advance after queue success. The child is the later host pump, not a call made by enqueue. Implementation PDF p5

Native service pump: poll, begin, dispatch, later reply

[Return to code diagram](#) | [Up: import](#) | [Up: export](#) | [Up: gui-startup / tick](#) | [Up: service-request / enqueue](#)

Supporting transport infrastructure called after native handle or tick. Only idle host state begins a queued request. The callee snippet shows normal request yield; shutdown/pending/current guards and epoch advancement precede it.

Scope: Supporting native service infrastructure

N6: Call provider with a completion callback	Callback may deliver selector cancellation/error. Worker content results are delivered by later UI polling. Browser hosts use Browser/WebSession callbacks instead.
--	---

complete_service: correlation, outcome, parser, publication

[Return to code diagram](#) | [Up: import](#) | [Up: export](#) | [Up: service-pump / poll](#) | [Up: native-transfer / poll](#)

The active-request guard rejects late/unmatched replies. The queue normalizes accepted status/text. Read-text success expands into the parser; accepted error/cancellation bypasses it. Browser and native delivery use this same shared method.

Scope: Shared application behavior

N1: Match active service, then advance epoch	Implementation PDF p104
N2: Complete and normalize queued result	Implementation PDF p104
N3: Handle error and cancellation	Implementation PDF p104
N4: Successful prompt or export result	Export reports handoff, not disk durability or that a browser user retained the offered download. Implementation PDF p104
N5: Successful read_text enters parser	Implementation PDF p104
N6: Catch parser failure, publish accepted outcome	The parser catch is included as source context. Accepted error/cancellation and prompt/export success also reach the same publication. Implementation PDF p104

Import parser: staged Store loop and atomic commit

[Return to code diagram](#) | [Up: import](#) | [Up: service-completion / import](#)

This is inside the successful read_text try block. Empty input reaches commit with no records. A trailing newline creates no extra record; an interior blank line is rejected by Store::add. Any exception returns to the parent catch without committing staged records or staged next-ID state.

Scope: Shared application behavior

N1: Copy Store and clear only the copy	Implementation PDF p104
N2: Check for another input line	Implementation PDF p104
N3: Find newline and borrow the line	Implementation PDF p104
N4: Strip trailing CR and validate/add	Implementation PDF p104
N5: Stop at end or advance beyond newline	Implementation PDF p104
N6: Commit all records and clear selection	Implementation PDF p104

Native content service: selector, worker, transfer and UI poll

[Return to code diagram](#) | [Up: import](#) | [Up: export](#) | [Up: service-pump / dispatch](#)

Supporting infrastructure excerpts, not application parsing. Read/write loops, bounds, temporary-file cleanup and destination replacement remain in the complete transfer source. The worker stores a value-only result; later UI polling returns it to Session. Browser file transport is a separate implementation.

Scope: Supporting native file-service infrastructure

N1: Selected path stays on the host	This callback follows a generic path prompt. Selector cancellation/errors use reply without starting the worker.
N4: Read branch calls the file reader	Regular-file and byte-bound checks precede/continue this fragment. Accepted chunks append to result.value. Implementation PDF p109
N5: Write branch calls the file writer	This writes an exclusive temporary file; successful close and noncancelled replacement commit afterward. It does not promise crash durability. Implementation PDF p109
N6: UI poll retrieves result and joins worker	Worker storage into result_ precedes this later poll. Session calls complete_service with the returned value; the worker never calls Application.